MCP 进入无状态时代：重构 Agent 的工具基础设施

孙校

Higress 社区

阿里云

CONTENT 目录

01 MCP 进入无状态时代

02 Higress：Agent 的 Tool Access Plane

03 Higress 的未来：从 MCP Gateway 到 Agent Gateway

阿里云

01 MCP 进入无状态时代

MCP 一直在摆脱连接状态

历史演进：逐步摆脱连接状态

最初 · 本地 stdio
Agent 与工具运行在同一环境

↓

2024-11 · HTTP + SSE
支持远程访问，但依赖双向连接

↓

2025-03 · Streamable HTTP
统一 Endpoint，长连接变为可选

↓

2026-07 · Stateless Core
请求独立处理，上下文随请求传递

2026-07-28：请求自描述

无握手 / 无 Session	server/discover 按需调用
上下文随请求	HTTP Header中携带请求元数据
任意实例可处理	tools/list 与 tools/call 独立调度

演进主线：工具调用逐步脱离特定连接，走向标准的远程请求

阿里云

无状态让 Tool 具备三种基础设施属性

Transport Session 退出核心，Tool 调用开始具备标准基础设施属性

01

状态边界

状态不再藏在连接里
业务状态需要通过合理设计显式表达

→

02

调度边界

任意请求可以落到任意健康实例
扩缩容与故障切换不再依赖粘性会话

→

03

治理边界

治理对象从 /mcp Endpoint
下沉到 Method 与具体 Tool

可调度 + 可识别 + 可治理 = 标准 Tool Workload

协议不会自动提供治理，但它让 Tool 具备了被共享基础设施治理的条件

阿里云

Agent 负责意图，基础设施守住边界



Agent 决定调用什么工具，基础设施保证调用可达、可控、可观测

阿里云

网关真正理解 MCP，才有资格治理 Tool

MCP Gateway = 协议语义 × 兼容转换 × 工具治理

看得懂

Version + Method + Tool Identity
协议识别与严格校验

接得住

Modern / Legacy / REST 的明确转换
升级不要求全链路同时完成

管得好

身份 + 凭证 + 策略 + 观测
治理对象下沉到具体 Tool

网关从代理 /mcp，走向理解协议版本、调用方法和 Tool 身份

阿里云

02 Higress：Agent 的 Tool Access Plane

阿里云

Higress：面向 AI 时代的云原生网关

统一管理 LLM API、MCP Server 与传统 API

开源 · CNCF Sandbox · Apache 2.0 · Wasm 可扩展

Higress

AI Native API Gateway

云原生底座 × AI 流量治理 × MCP Server 托管

AI Gateway　MCP Gateway

github.com/higress-group/higress

01 统一接入	LLM API · MCP Server · REST / 企业服务
02 统一治理	路由与弹性 · 认证鉴权 · 限流 · 观测审计
03 开放底座	Istio + Envoy · Wasm 插件 · Kubernetes / Gateway API

在 Agent 与 Tool Runtime 之间，提供一层可统一管理的 Tool Access Plane

阿里云

企业 Agent 上线，通常先卡在三道门

目标 | 可规模化上线

企业 AI 平台团队

目标：让业务 Agent
调用现有系统并进入生产

01 协议升级

Client 已升级
Server 仍依赖旧生命周期

02 工具接入

REST、OpenAPI 与 MCP Server
形态和入口各不相同

03 执行治理

凭证、权限、审计分散

工具可用 | 升级可控 | 调用可追踪

Higress 逐一消除协议、接入与治理摩擦

新 Client 不等全链路升级，也能调用旧 Server

调用侧 | 2026-07-28 Client

自描述 tools/call
无握手 · 无 Session
版本与 Tool 身份随请求携带

Higress 协议桥

01 识别校验
版本 · 方法
Tool 身份

02 显式转换
protocolStrategy
legacy

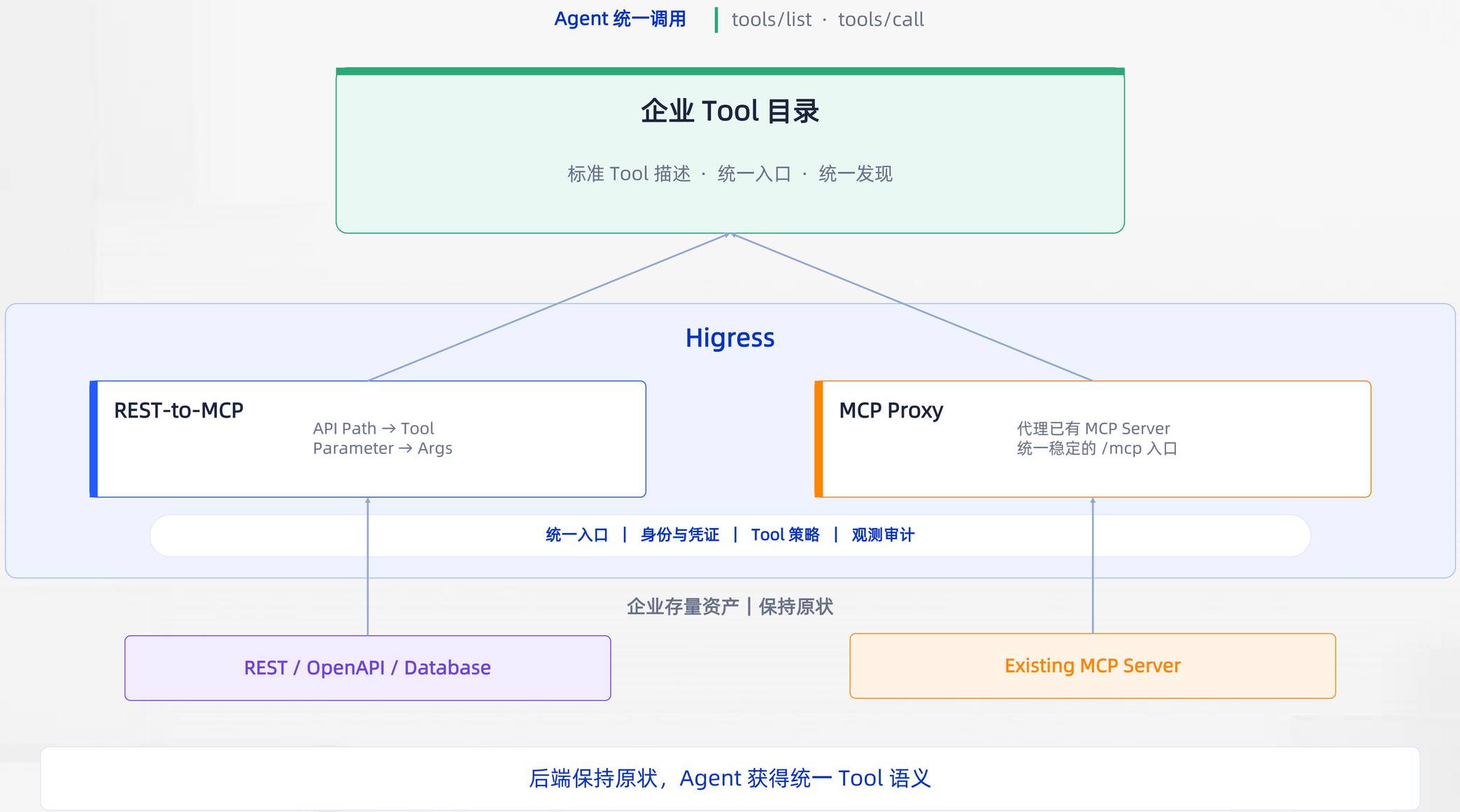
03 调用安全
不猜版本
不跨协议重试

后端 | Legacy MCP Server

Legacy MCP 生命周期
initialize · Session
SSE / Streamable HTTP

客户端先升级 | 服务端渐进迁移 | 一次调用只执行一次

存量系统不改造，也能进入统一 Tool 目录



后端保持原状，Agent 获得统一 Tool 语义

阿里云

Skill + CLI 能执行，企业仍需 Tool 治理边界

能运行命令，不等于可以安全共享给多个 Agent

直接执行 | Skill → Shell / CLI → 企业系统		受控执行 | Skill → MCP Tool → Higress → 企业系统
Shell 权限过宽 业务动作与运行环境没有分离	01	**调用侧身份** 识别 Agent、用户与租户
Token 与版本散落 难以统一吊销、轮换和升级	02	**凭证隔离** Client Token 与后端 Credential 分离
命令字符串审计 缺少业务 Tool 语义	03	**Tool 策略与观测** 白名单 · 限流 · 指标 · Trace · 审计

审计答案：谁 · 调用哪个 Tool · 使用哪份凭证 · 执行结果

Skill 决定怎么做 | MCP 定义能做什么 | Higress 守住谁能做、如何执行

阿里云

03 Higress的未来：从 MCP Gateway 到 Agent Gateway

MCP Gateway 只覆盖了 Agent 的一条访问链



问题不是增加三套网关，而是把四类访问纳入同一个治理边界

Higress Agent Gateway：目标形态与能力演进

Agent Runtime（Agent 侧）
Planner · Memory · State
业务状态属于 Agent

Higress 能力边界

● 已具备 | ● 待实现

Agent 服务入口

网关入口 · Streaming · 认证底座
Agent API · 身份 · 服务发现

模型访问

模型路由 · Fallback
Token / 配额治理
Agent 级成本与策略语义

Higress Agent Gateway

工具访问

MCP Gateway · REST-to-MCP
凭证隔离 · Tool 策略 / 审计
工具生命周期治理

Agent 间协作

路由 · 安全 · 观测底座
A2A · Agent Card · Skill · Task

通用网关能力：身份 · 路由 · 限流 · 观测 · 审计
Agent 身份 · 策略 · 调用链 · 成本语义

Agent Runtime 保持在 Agent 侧，Higress 在 AI Gateway 与 MCP Gateway 上补齐统一访问、治理与协作能力



二维码有效期至 8 月 25 日

github.com/higress-group/higress

欢迎参与：Samples · Conformance · Issue · PR

阿里云 | 计算，为了无法计算的价值

阿里云

基于 AgentScope 2.0 Harness 构建企业级、分布式智能体

刘军

阿里云

CONTENT
目录

01 AgentScope 2.0 介绍

02 AgentScope Harness 核心设计

03 AgentScope Service 企业级智能体控制面

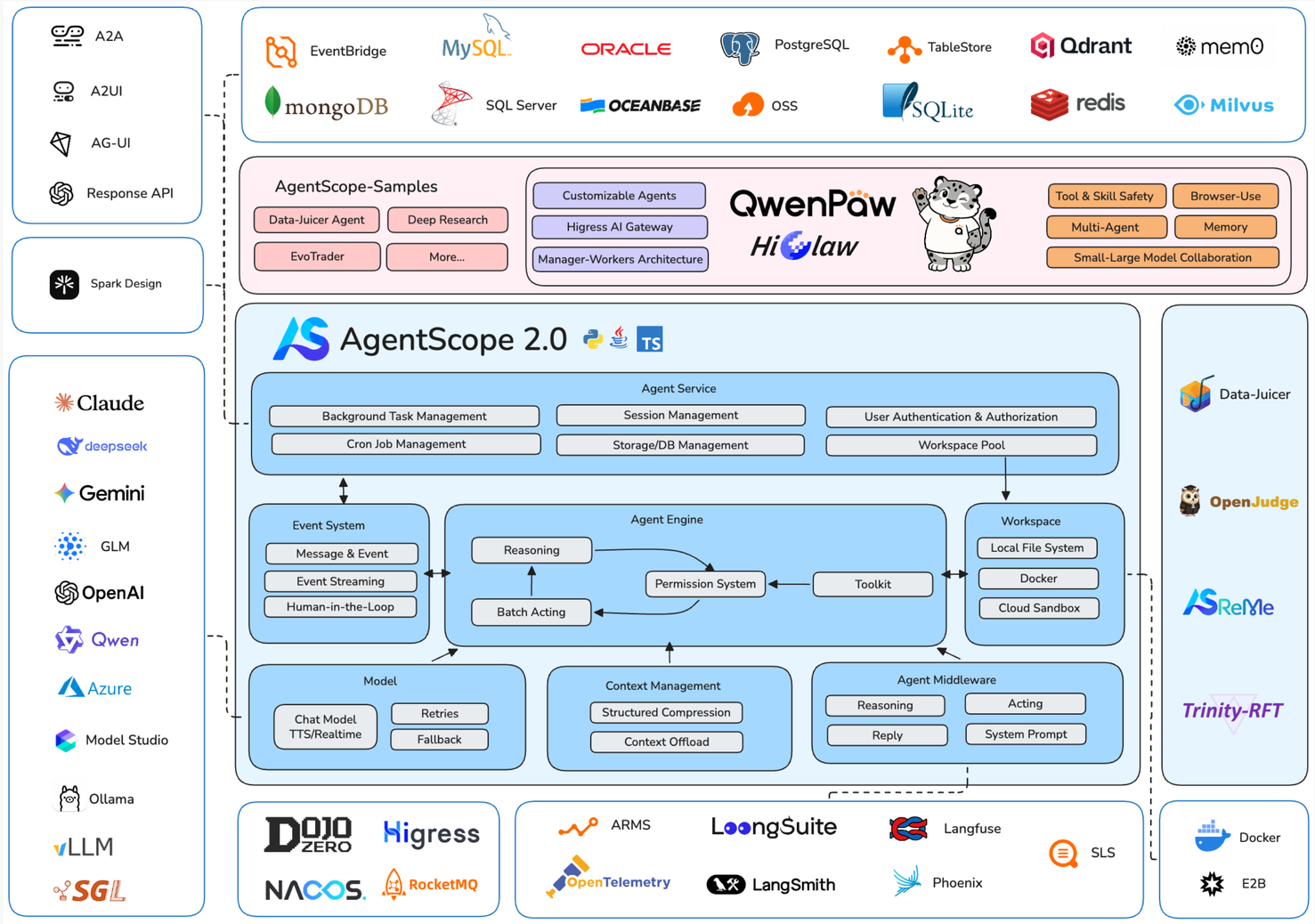
阿里云

01 AgentScope 2.0 介绍

AgentScope 概览

AgentScope 是阿里巴巴推出的一款面向企业级、分布式场景的智能体开发框架，覆盖 Python、Java、Typescript、Golang（建设中）语言实现。

AgnetScope 生态核心目标：提供一套覆盖“开发、部署、调优”全生命周期的生产级解决方案，让智能体应用的开发更简单、运行更稳定、效果更卓越。



阿里云

ReactAgent 核心组件

Agent 智能体

- ReAct 推理-行动循环引擎
- 多用户 / 多会话并发安全
- 流式事件 & 结构化输出
- 中断 / 恢复 & 人机交互

Model 模型层

- Credential + ChatModel 两层架构
- 5大厂商: DashScope / OpenAI / Anthropic / Gemini / Ollama
- Streaming & Thinking & 多模态
- 可扩展自定义 Provider

Tool 工具系统

- @Tool 注解 / ToolBase 继承
- MCP 协议集成 (STDIO / SSE / HTTP)
- Skill 热加载 Markdown 指令集
- Tool Group 按需激活/自管理

Context 上下文

- 无状态引擎 + AgentState 持久化
- RuntimeContext per-call 元数据
- Redis / MySQL 分布式状态共享
- 跨节点故障转移 & 会话恢复

Middleware 中间件

- 5 个生命周期 Hook 位置
- 洋葱式(Onion) + 变换式(Transformer)
- OpenTelemetry 全链路追踪
- 限速 / 回退 / 动态 Prompt

Permission 权限

- Rules + Mode + Built-in Checks
- 5 种模式: DEFAULT / EXPLORE / BYPASS / ACCEPT_EDITS / DONT_ASK
- 建议规则自动生成 & 持久化
- 危险路径不可绕过保护

Message & Event

- Msg 类型化内容块体系
- AgentEvent 流式增量传输
- Start → Delta → End 生命周期
- SSE 推送 & 断点重建消息

阿里云

1.0 -> 2.0 迁移指南

✓ 总体兼容 — 平滑升级

核心 API 保持一致
ReActAgent Builder 模式不变，model / toolkit / sysPrompt 等参数平滑迁移

@Tool 注解完全兼容
已有 @Tool / @ToolParam 标注的工具类无需任何修改即可在 2.0 中注册使用

MCP 客户端无缝衔接
McpClientBuilder API 不变；STDIO / SSE / Streamable HTTP 三种连接方式保持兼容

Model 层 Provider 兼容
DashScope / OpenAI / Anthropic / Gemini / Ollama 各 ChatModel builder 接口不变

消息体系向后兼容
UserMessage / AssistantMessage / SystemMessage 构造方式不变

✗ 破坏性变更 — 必须改造

状态管理架构重构
无状态 Agent 引擎：AgentState 按 (userId, sessionId) 寻址，不再绑定单一 agent 实例

RuntimeContext 替代旧调用模式
call() 必须传入 RuntimeContext；per-call 元数据不再挂在 agent 实例上

AgentStateStore 强制配置
分布式部署(sandbox)下必须配置 Redis / MySQL 等分布式状态后端，否则 build() 抛异常

Middleware 注册方式变更
新增 5 层 hook 体系；旧回调/拦截器需迁移为 MiddlewareBase 实现

Permission System 全新引入
工具执行前置权限检查为必选项；需配置 PermissionContextState（至少选择 Mode）

⚠ 已废弃 — v2.1 将移除

Memory 接口
InMemoryMemory / LongTermMemory 等标记 @Deprecated(forRemoval=true)

→ 迁移到 AgentState.getContext() + AgentStateStore

ToolExecutionContext
标记 @Deprecated；底层自动桥接到 RuntimeContext，老代码暂不失效

→ 迁移到 RuntimeContext

ToolCallParam.getContext()
已废弃

→ 迁移到 getRuntimeContext()

ImageBlock / AudioBlock / VideoBlock
仍兼容但新代码建议统一使用 DataBlock

→ 迁移到 DataBlock + mediaType

旧 interrupt() 无参重载
单 session 场景仍有效，多 session 下行为不确定
→ 迁移到 interrupt(userId, sessionId)

阿里云

02 AgentScope Harness 核心设计

阿里云

AgentScope Harness 总体架构

在 ReActAgent 之上，把长期运行 agent 必备的工程能力打包

应用 · 用户请求

HarnessAgent

薄包装 · 能力叠加在 ReAct 循环的关键时机 · 内部 middleware 顺序固定

Workspace
人格 / 知识

Memory
双层长期记忆

Compaction
上下文压缩

Subagents
子 agent 编排

Filesystem
本机/KV/沙箱

Skills
技能装配

Plan Mode
只读思考 +HITL

Channel / Gateway
会话 · 路由 · SSE

Permission
工具白名单

ReActAgent · 推理循环（core）· Harness 不改写此层，只叠加钩子

共享对象（能力间零耦合，只通过这三者通信）

RuntimeContext
userId · sessionId · extra

Workspace
AGENTS.md · MEMORY · skills

AgentStateStore
跨请求恢复运行时状态

AgentScope Harness 快速体验

```xml
<dependency>
    <groupId>io.agentscope</groupId>
    <artifactId>agentscope-harness</artifactId>
    <version>${agentscope.version}</version>
</dependency>
```

```java
public class FirstAgent {
    public static void main(String[] args) {
        HarnessAgent agent = HarnessAgent.builder()
                .name("note-taker")
                .sysPrompt("你是一个帮助用户做笔记的助手。")
                // 字符串形式由 ModelRegistry 解析 —— 自动读取 DASHSCOPE_API_KEY;
                // 切换其他厂商时改用 "openai:gpt-5.5"、"anthropic:claude-sonnet-4-5"、
                // "gemini:gemini-2.0-flash" 或 "ollama:llama3"。
                .model("dashscope:qwen-plus")
                .workspace(Paths.get(".agentscope/workspace"))
                .compaction(CompactionConfig.builder()
                        .triggerMessages(30)
                        .keepMessages(10)
                        .build())
                .build();

        RuntimeContext ctx = RuntimeContext.builder()
                .sessionId("demo-session")
                .userId("alice")
                .build();

        // 第一轮：自我介绍 + 当天的事
        agent.call(new UserMessage("我叫天宇，今天准备一个关于 ReAct 的技术分享。"), ctx).block();

        // 第二轮：同 sessionId，自动恢复上一轮状态后回答
        agent.call(new UserMessage("我叫什么？我今天要干什么？"), ctx).block();
    }
}
```

Workspace -- 智能体进化的 Source of Truth

「智能体是什么」+「学到了什么」都是文件

内容按生命周期分三类

静态资产

工程师编辑

`AGENTS.md · knowledge/ · skills/ · subagents/ · tools.json`

运行时文件

框架 / agent 写

`agents/<id>/sessions/ · agents/<id>/tasks/ · plans/`

长期记忆

agent + 后台任务

`MEMORY.md · memory/YYYY-MM-DD.md`

四条设计线索

① **定义与进化都是文件**

不散落代码、不绑数据库；一个目录拷走 = 完整 agent

② **生命周期三分**

静态 / 运行时 / 长期记忆 · 走不同读写路径

③ **原生多租户隔离**

IsolationScope: SESSION / USER / AGENT / GLOBAL

④ **Workspace ⊥ Filesystem**

同一份目录布局 → 三种物理后端可切换

抽象文件系统 – Workspace 的物理载体

同一份逻辑目录，三种物理后端，agent 代码零改动

Workspace · 逻辑目录布局

AbstractFilesystem 接口 · WorkspaceManager 路由

Local +Shell

默认

✓提供 Shell

- Overlay: workspace +project
- 路径策略 ROOTED / SANDBOXED
- 单进程 · 本机开发 · 信任环境
- 快、简单、无外部依赖

Remote KV

多副本

✗ 不提供 Shell

- Redis · JDBC · OSS · Nacos
- 本机模板 + 远端覆盖（两层读）
- 多副本共享 MEMORY / sessions
- 管理台改文件下轮生效

Sandbox

隔离

✓容器内 Shell

- Docker · K8s · E2B · Daytona · AgentRun
- Workspace Projection（SHA-256 增量）
- 快照恢复 pip / npm install 状态
- 生产跑不可信代码首选

阿里云

内置上下文压缩策略 - 四道防线

让对话保持在 token 预算内，同时不丢关键信息

压缩流水线（按触发时机排布）

Tool 执行	→	Result Eviction	→	Truncate Args	→	Summary LLM	→	Overflow Recover
产生工具结果		单条 > 80K 落盘 上下文留首尾 +指针		大参数字符串截断 零 LLM 成本		前缀 → 结构化摘要 尾部保留原文		真的撞墙 → 极端压缩 自动重试一次

可调节的杠杆

触发阈值	`triggerMessages · triggerTokens`
保留窗口	`keepMessages · keepTokens`
Flush 时机	`ALWAYS · NEVER · THROTTLED`
独立小模型	`compaction.model / memory.model`
卸载排除	`read_file · grep_files 默认排除`

压缩不会触碰的内容

Plan Mode 状态	`AgentState.planModeContext`
子 agent 后台任务	`agents/<id>/tasks/<sid>.json`
todo_write 清单	`AgentState.tasksContext`
权限规则	`AgentState.permissionContext`
永不压缩对话日志	`sessions/<sid>.log.jsonl`

双层长期记忆- 事实自动沉淀

让 agent 记住跨会话的事实，同时避免上下文无限膨胀

对话 messages
当前上下文

Flush LLM
每次 call 结束

第一层 · 日流水账
memory/YYYY-MM-DD.md · 追加 · 未去重

Consolidation LLM
后台节流 · 最少 30 min/次

第二层 · MEMORY.md
合并去重 · 策划后的长期记忆

每轮注入

System Prompt
<memory_context>

agent 主动查询工具

memory_search | memory_get | session_search

见到 MEMORY.md 截断提示时会自动调用

管线里的三次独立 LLM 调用（定制入口）

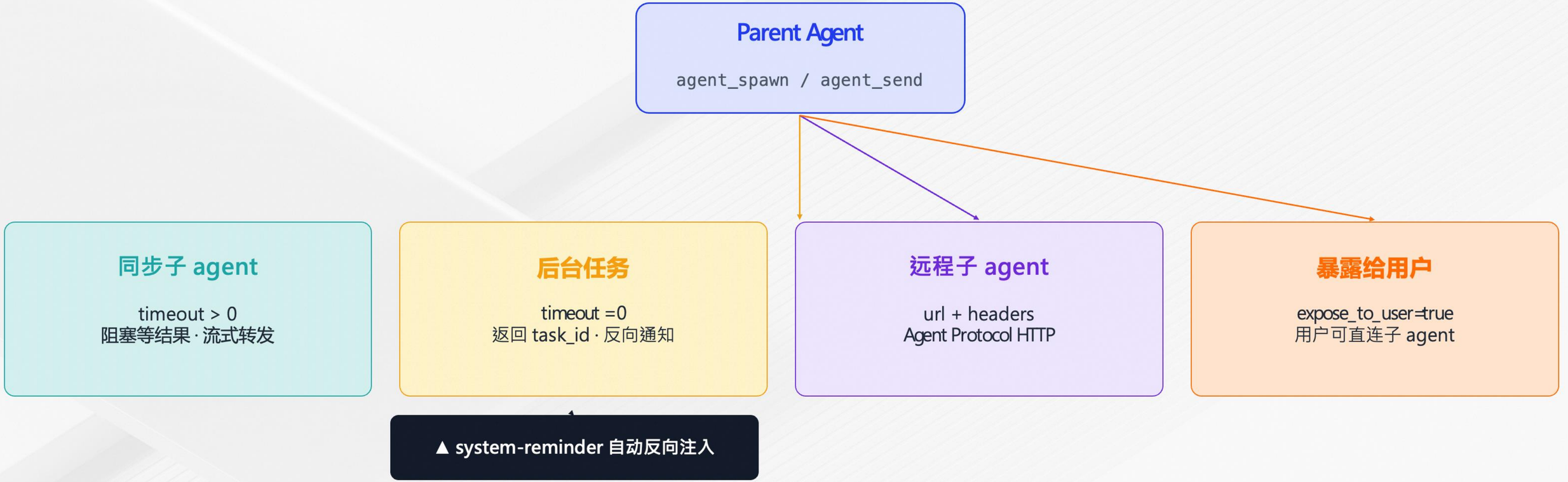
Flush
`MemoryConfig.flushPrompt`
从对话窗口抽取长期事实到日流水账

Consolidation
`MemoryConfig.consolidationPrompt`
把每日流水账合并去重到 MEMORY.md

Compaction Summary
`CompactionConfig.summaryPrompt`
把对话前缀压成一条摘要（当下上下文）

子智能体编排、委派、并行、异步通知

把独立、上下文重、可并行的任务交给专家型子 agent



关键能力

- ISOLATED / SHARED workspace
- 流式事件带 source 路径
- persistSession 复用实例
- 递归 3 层硬上限
- DENY 权限自动继承
- Plan Mode 自动只读

阿里云

沙箱管理：隔离、恢复与分布式

把危险操作关进容器，把安装状态存进快照

01 执行边界

文件与 shell 命令都在隔离容器里执行，宿主完全不参与

02 跨调用恢复

pip install · npm install · 临时文件都随快照保留，下次 call 无需重装重装

03 多副本可用

通过分布式 store +远端快照，任意节点都能 resume 出同一份工工作区

快照后端

- Local · 本地磁盘
- Redis · 低延迟
- OSS/S3 · 多副本
- JDBC · 关系库

call() 生命周期 → 沙箱决策



有 → 复用

最快路径

无容器 有快照 → 恢复

快照重建

都没有 → 冷启动

WorkspaceSpec 全量初始化

后端矩阵 · Docker · Kubernetes · Daytona · E2B · AgentRun

IsolationScope 决定沙箱 slot 复用（USER 默认 · SESSION 天然并发安全 · AGENT/GLOBAL 需 executionGuard 串行化）

Skills：四层注册中心 & 沙箱内执行

SKILL.md + references/ + scripts/ = 可复用的能力包

注册中心 · 四层优先级（低 → 高）

L4 — <userId>/skills/

用户级隔离目录 · 覆盖共用版

L3 — workspace/skills/

工作区共用 · 项目特有约定

L2 — skillRepository(...)

市场后端 · Git / Nacos / MySQL / Classpath

L1 — projectGlobalSkillsDir

项目全局目录 · ~/.agentscope/skills/

▲ 高

▼ 低

沙箱内执行 · 三步透明化

01 物化 Stager

市场 skill 从内存写到宿主 .skills-cache/
· SHA-256 文件级去重
· 启发式恢复 chmod +x

02 Workspace Projection

把 skills/ + .skills-cache/ 打成 tar
· hydrate 进沙箱 /workspace
· 内容 SHA-256 整体比对增量

03 容器内执行

<files-root> 用容器内绝对路径
· agent 不用猜路径
· 快照保留 pip/npm install 副作用

阿里云

计划模式：想清楚 -> 写下来 -> 再动手

只读思考阶段 +计划文件 +HITL 退出

工作流（明确固化四步：设计 → 写计划 → 人确认 → 执行）



Plan 阶段的白名单

只读文件工具	`read_file` · `grep_files` · `glob_files` · `list_files`
内存查询工具	`memory_search` · `memory_get` · `session_search`
Plan 三件套	`plan_enter` · `plan_write` · `plan_exit`
任务清单	`todo_write`
Shell（可选）	`.allowShellInPlanMode()` 后放开为只读用途

四种终态（别只看 isPlanModeActive）

未进入	模型直接 build，任务与 workspace 不匹配
进入 → plan_exit	成功：规划完 +获批 → build 模式
仍 plan · 有 PLAN	已起草但未退出；后续消息可批准继续
仍 plan · 无 PLAN	只说不做：文本像计划但没写出

Channel：消息平台 -> Gateway -> Agent

一行接线，自动完成会话管理 · 并发 · 路由 · 流式

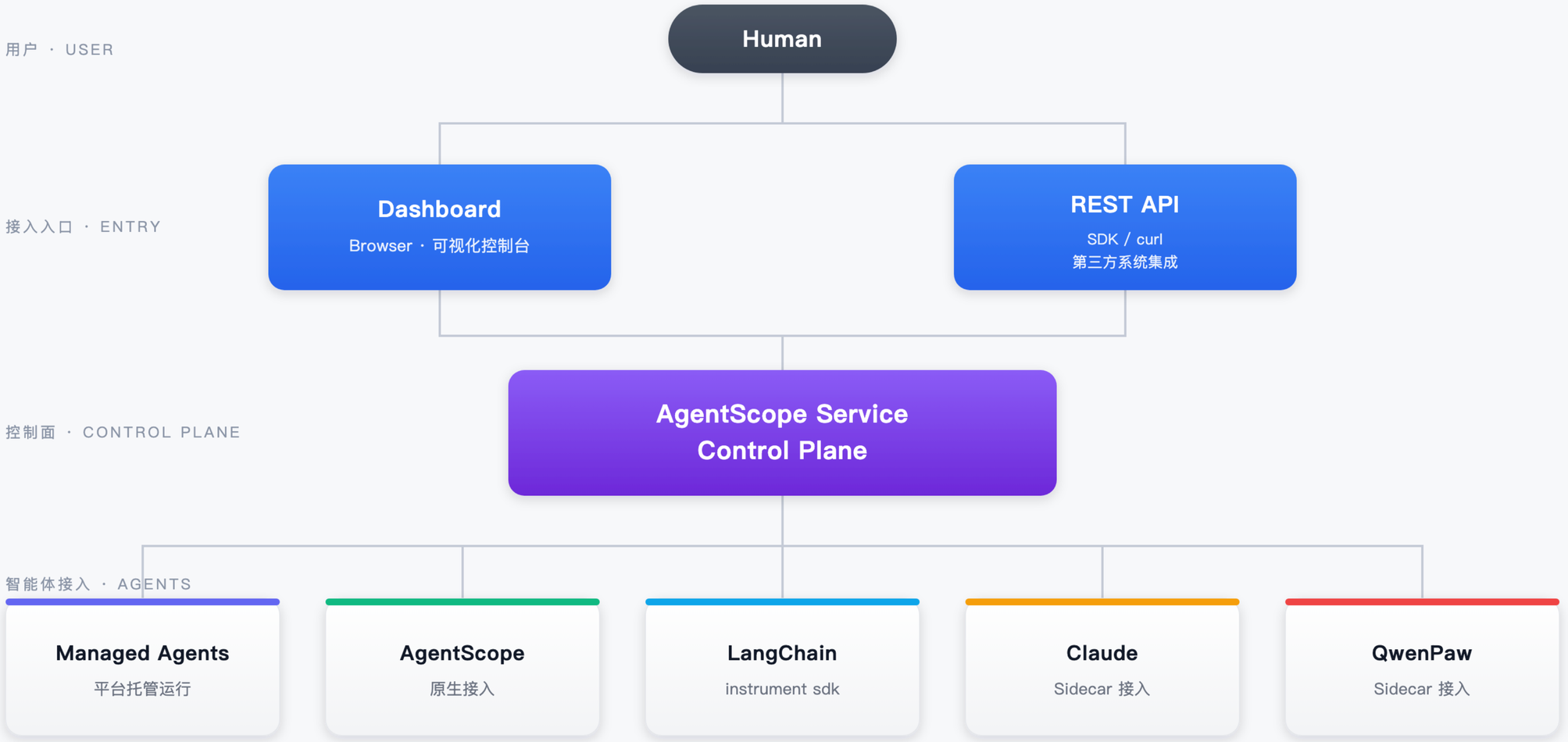


阿里云

3 AgentScope Service 企业级智能体控制面

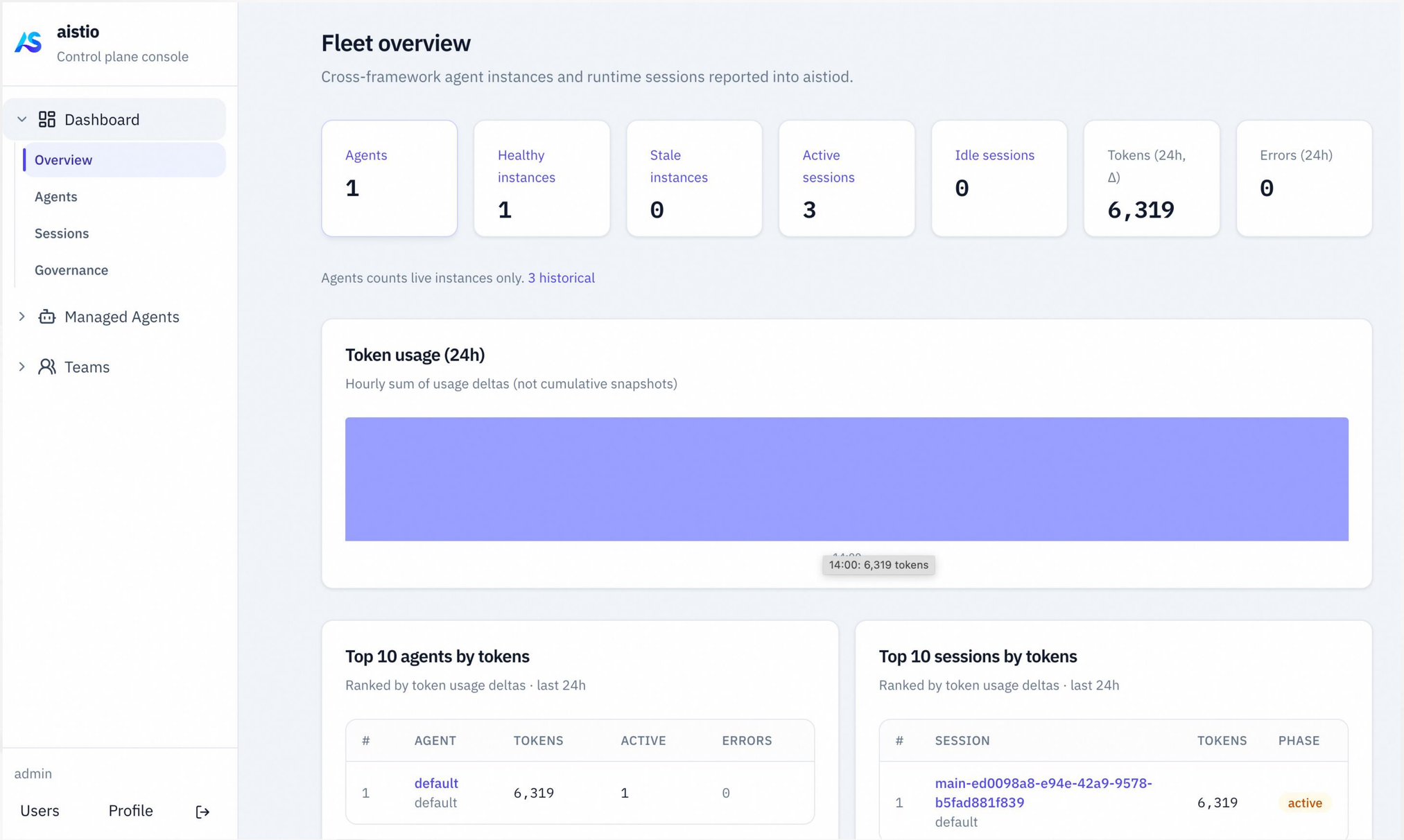
AgentScope Service -企业级智能体编排控制面

AgentScope Service 的目标不是替换你现有的 Agent 框架，而是提供一层统一控制面，让你把用不同框架（AgentScope、Langchain、ADK 等）、不同技术栈（Claude、OpenClaw、QwenPaw 等）搭建的智能体统一管控、协作起来。



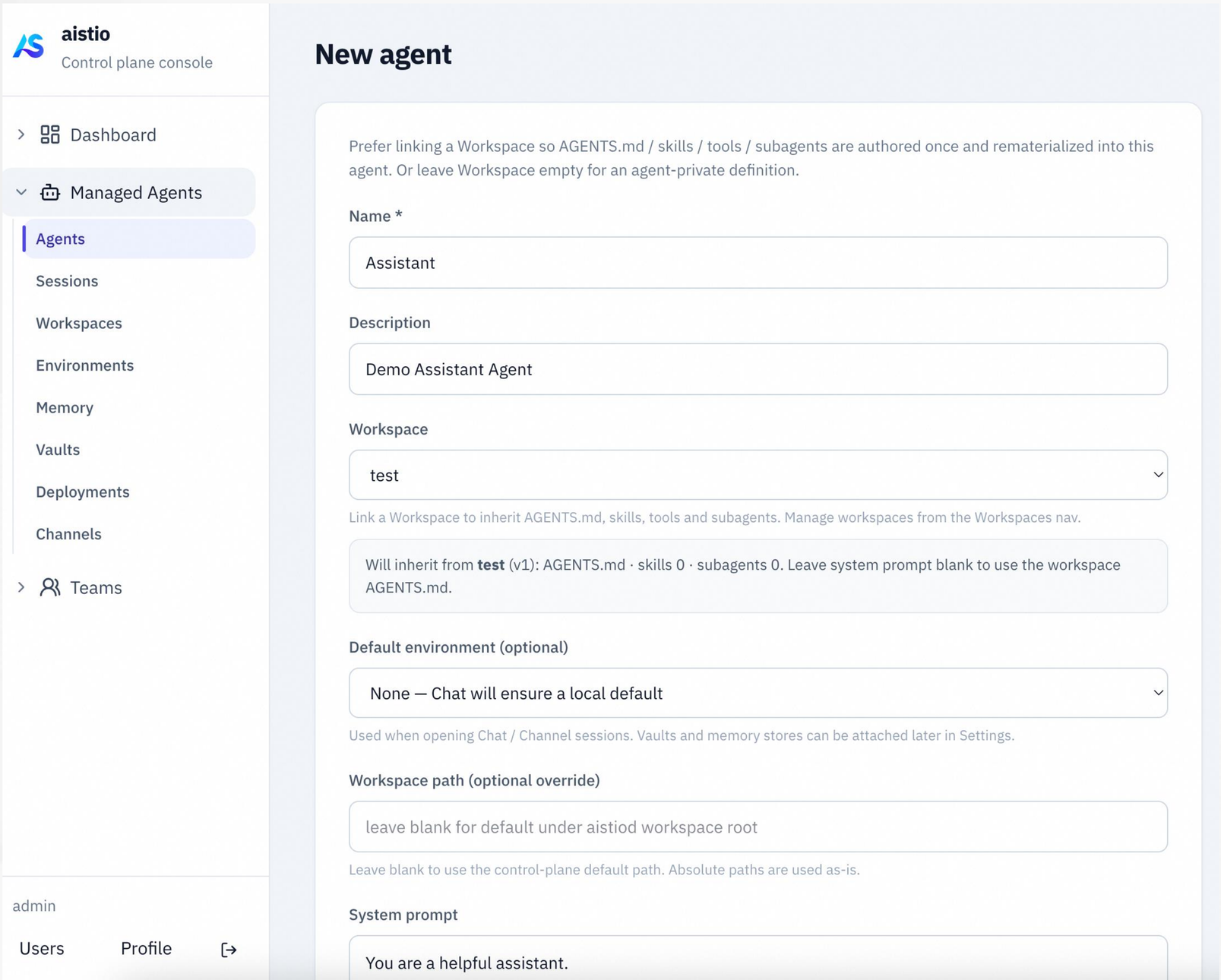
AgentScope Service · 系统架构图

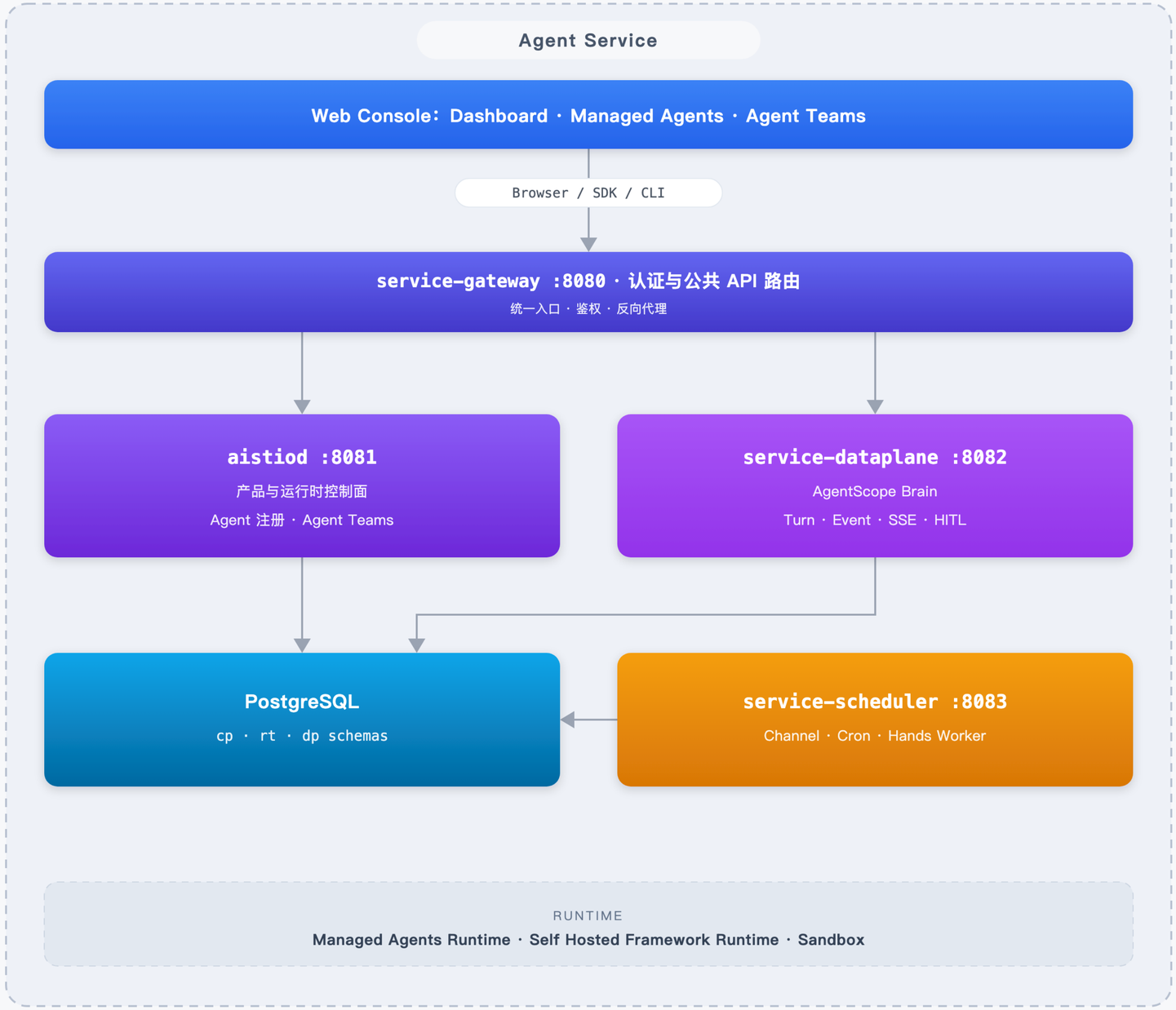
控制面 & Dashboard



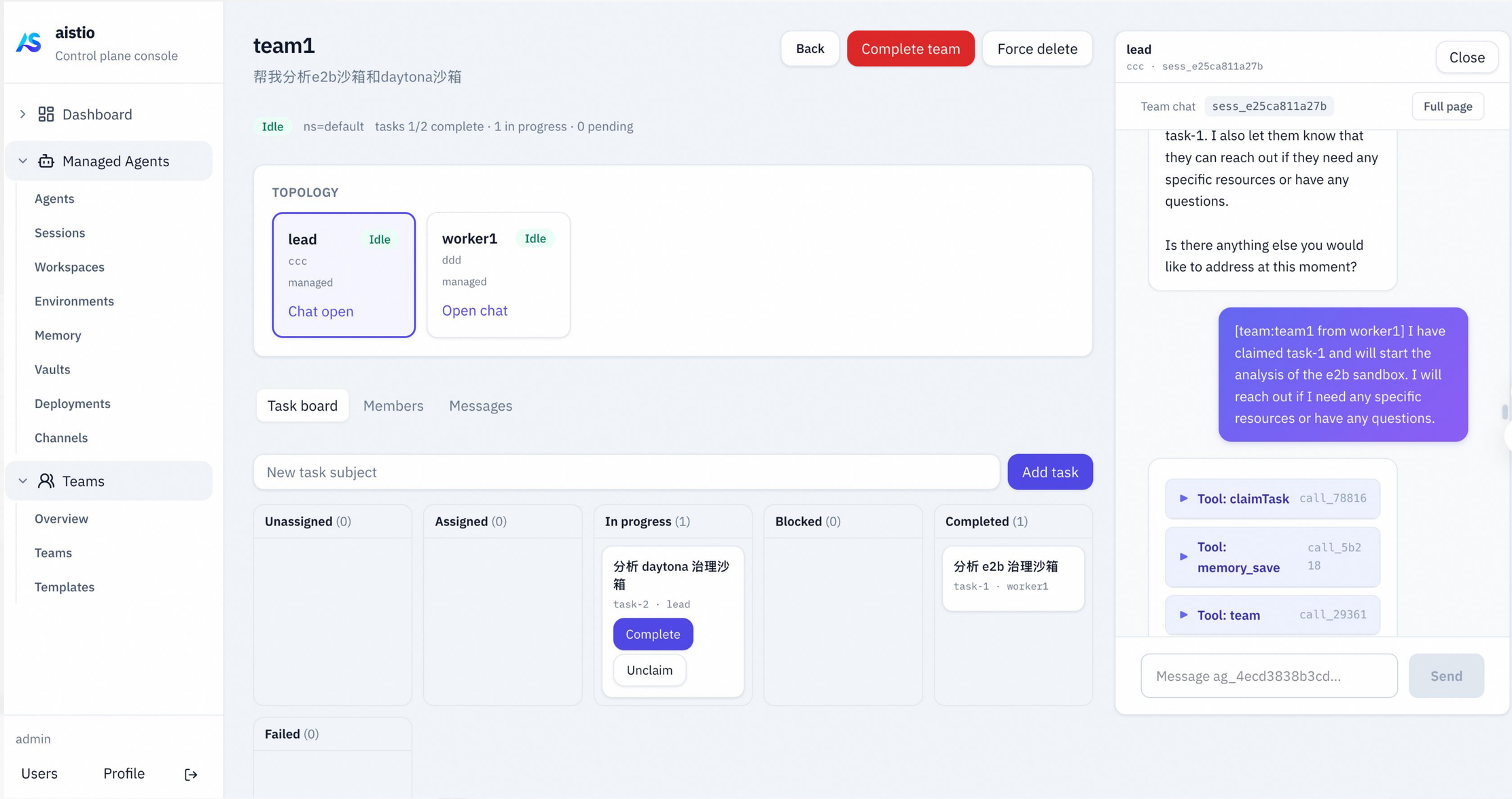
阿里云

Managed Agents 实现 Harness 全托管




Agent Service
Web Console：Dashboard · Managed Agents · Agent Teams
Browser / SDK / CLI
service-gateway :8080 · 认证与公共 API 路由
统一入口 · 鉴权 · 反向代理
aistiod :8081
产品与运行时控制面
Agent 注册 · Agent Teams
service-dataplane :8082
AgentScope Brain
Turn · Event · SSE · HITL
PostgreSQL
cp · rt · dp schemas
service-scheduler :8083
Channel · Cron · Hands Worker
RUNTIME
Managed Agents Runtime · Self Hosted Framework Runtime · Sandbox

实现多 Agent 协作


aistio
Control plane console
Dashboard
Managed Agents
Agents
Sessions
Workspaces
Environments
Memory
Vaults
Deployments
Channels
Teams
Overview
Teams
Templates
admin
Users
Profile
team1
帮我分析e2b沙箱和daytona沙箱
Back
Complete team
Force delete
Idle
ns=default
tasks 1/2 complete · 1 in progress · 0 pending
TOPOLOGY
lead
Idle
ccc
managed
Chat open
worker1
Idle
ddd
managed
Open chat
Task board
Members
Messages
New task subject
Add task
Unassigned (0)
Assigned (0)
In progress (1)
分析 daytona 治理沙箱
task-2 · lead
Complete
Unclaim
Blocked (0)
Completed (1)
分析 e2b 治理沙箱
task-1 · worker1
Failed (0)
lead
ccc · sess_e25ca811a27b
Close
Team chat
sess_e25ca811a27b
Full page
task-1. I also let them know that they can reach out if they need any specific resources or have any questions.
Is there anything else you would like to address at this moment?
[team:team1 from worker1] I have claimed task-1 and will start the analysis of the e2b sandbox. I will reach out if I need any specific resources or have any questions.
Tool: claimTask
call_78816
Tool: memory_save
call_5b218
Tool: team
call_29361
Message ag_4ecd3838b3cd...
Send

扫码加入钉钉群

阿里云 | 计算，为了无法计算的价值

ROCKETMQ · LITETOPIC

LiteTopic 实践案例

轻量队列模型特性解析与场景落地

百炼平滑限流案例



CONTENTS

目录

01 **LiteTopic 特性**
轻量队列模型的四个核心特性详解

02 **百炼平滑限流案例**
从突发限流到通用限流插件

LITETOPIC · OVERVIEW

LiteTopic 特性

面向海量短生命周期会话的轻量队列模型，四个核心特性逐一拆解

01

PART 01 · 什么是 LITETOPIC

依然是发布 / 订阅模型

LiteTopic 与传统 Topic 本质相同——都是基于主题的 Pub/Sub 异步消息模型



共同本质

异步解耦 · 削峰填谷

基于主题的发布 / 订阅

Message → Topic → Consumer

区别在于：LiteTopic 在模型细节和使用场景上有自己的特点——更轻量的生命周期管理、更大规模的队列支撑、更灵活的订阅方式。接下来逐一展开。

PART 01 · 特性一

动态创建与销毁

不需要预配置，业务运行时数据面决定队列名字，随用随建、用完即走

传统 Topic

- 预先创建，生命周期长
- 与应用/业务生命周期一致
- 如交易系统中的 "Order" Topic
- 创建需审批、配置路由、分配资源

LiteTopic

- 动态创建和生成，运行时即建即用
- 业务数据面决定其名字
- 一个 SessionId、TaskId、RequestId 均可
- 粒度可粗可细，由业务场景决定

Session-A　Task-42　Request-xyz　Session-B　… 随时创建 …

PART 01 · 特性二

低成本 · 海量规模

从数千到千万级的跨越，TTL 回收策略保障资源不泄漏

传统 Topic 上限（商业实例）

3,000

受实例规格限制，创建成本高

LiteTopic 支撑规模

20W ~ KW

轻量化设计，创建与维护成本极低

TTL 自动回收

当某个 LiteTopic 不活跃时间超出配置后，自动销毁回收，释放 Quota 空间。无需人工清理，资源零泄漏。

PART 01 · 特性三

差异化订阅

打破 Group 集群消费的限制，每个客户端自由组织订阅关系

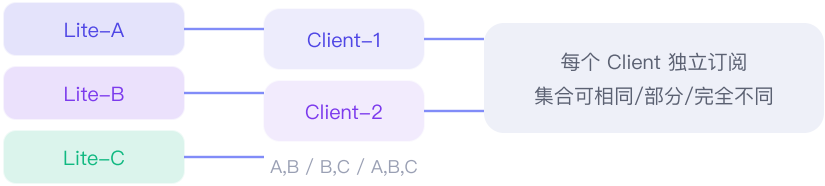
传统消费模型

基于 Group 分组的集群消费：相同订阅主题、相同处理逻辑的一组消费实例通过 Group 归组，一条消息在集群内被其中一个实例消费。



Lite 消费模型

单个客户端维度添加自己感兴趣的主题。多个客户端可订阅相同集合，也可部分相同、完全不同，灵活组织订阅关系。



PART 01 · 特性四

顺序消费

单个 LiteTopic 天然 FIFO，需要并发时可拆分队列

传统顺序消息

同一 Topic 下多个逻辑队列，ShardingKey（业务 ID）Hash 决定落入哪个队列。同一业务 ID 的消息一定在同一队列上保持顺序，但不同业务 ID 可能共享同一队列。



同 ID 必同队列（保序），不同 ID 可共享队列（交错排列）

LiteTopic 顺序

一个 LiteTopic 全局只有一个逻辑队列，默认就是 FIFO 消费。队列内全部是同一个 LiteTopic 的消息，天然有序，无需 ShardingKey。



CASE 1 · BAILIAN PLATFORM

百炼平滑限流案例

用 RocketMQ LiteTopic 重构大模型限流：从"直接拒绝"到"排队放行"

02

PART 02 · 2.1 背景

百万租户 × 稀缺 GPU，限流误差被成倍放大

1

百万级租户抢算力

百万级用户同时调用数十种大模型，GPU 是最稀缺的生产要素。

2

误判即双向损失

限流误判：用户超时、频繁重试；过载放行：算力被浪费、他人体验劣化。

3

误差被百万租户放大

一次误判的影响会沿租户规模扩散，小误差酿成大故障。

PART 02 · 2.2 百炼突发限流的特点

从"门口直接拦人"升级为"取号等候区"

传统限流（两态）

- 放行：在阈值内立即通过
- 拒绝：超出即返回 429
- 突发流量被直接误伤
- 只能"拒绝"，不能"缓冲"

平滑限流（三态）

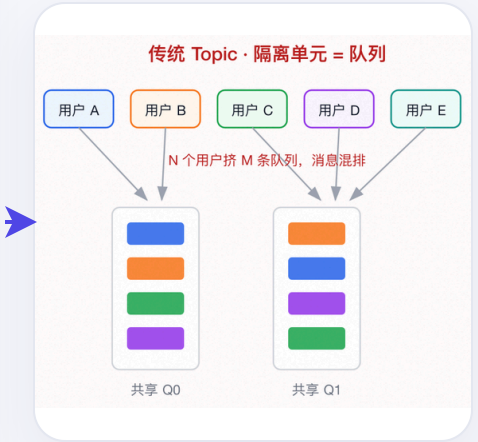
- PASS：能过就立即过
- QUEUE：过不了就排队等候
- REJECT：排满了才拒绝
- 用异步队列削峰填谷

排队等一等，好过直接被拒之门外

PART 02 · 2.3 基于 RocketMQ 的方案

限流识别 → MQ 缓冲入队 → 排队整形 → 模型服务





队头阻塞

传统消费模型自旋时HOL Blocking;
每个【用户+模型】需创建独立 `Topic/Group` 保证互不影响

运维成本高 · 无法合池

独立消费集群，无法跨用户/模型共享
资源利用率低

缺乏扩展性

特殊用户特殊配置，无法动态调整
新增用户或模型需新增配置，`topic`以`userId + model`命名

PART 02 · 2.4 落地效果

四个基于 LiteTopic 的适配

1 海量

为每个 userId、userId + model、APIkey 创建独立队列，精准限流。

2 动态

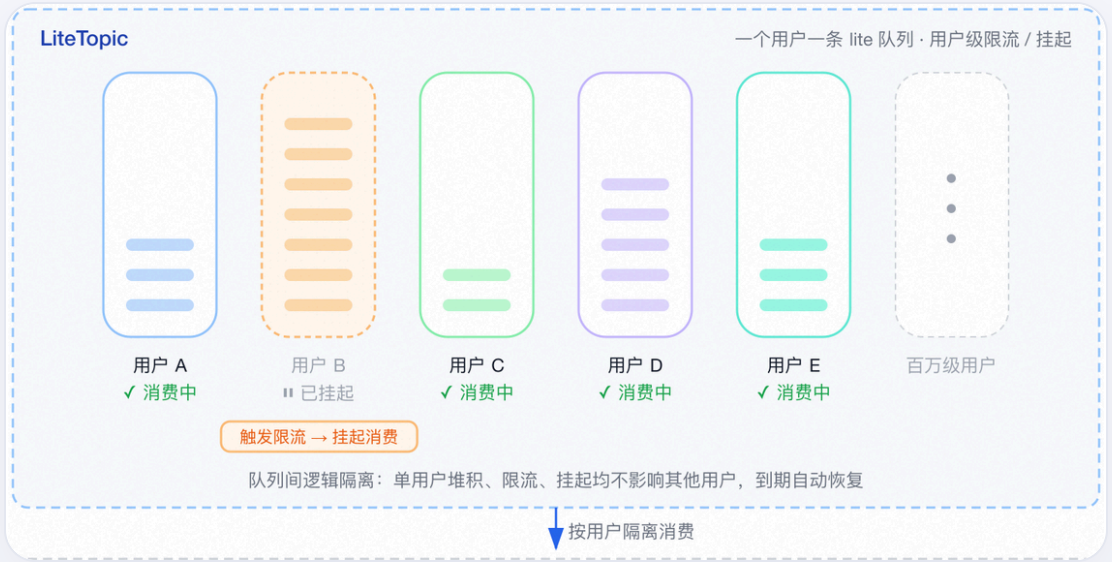
需要开启的用户，直接 Diamond 配置，入口处生效。

3 Suspend 语义

按队列挂起，毫秒到小时级控制。

4 精准订阅

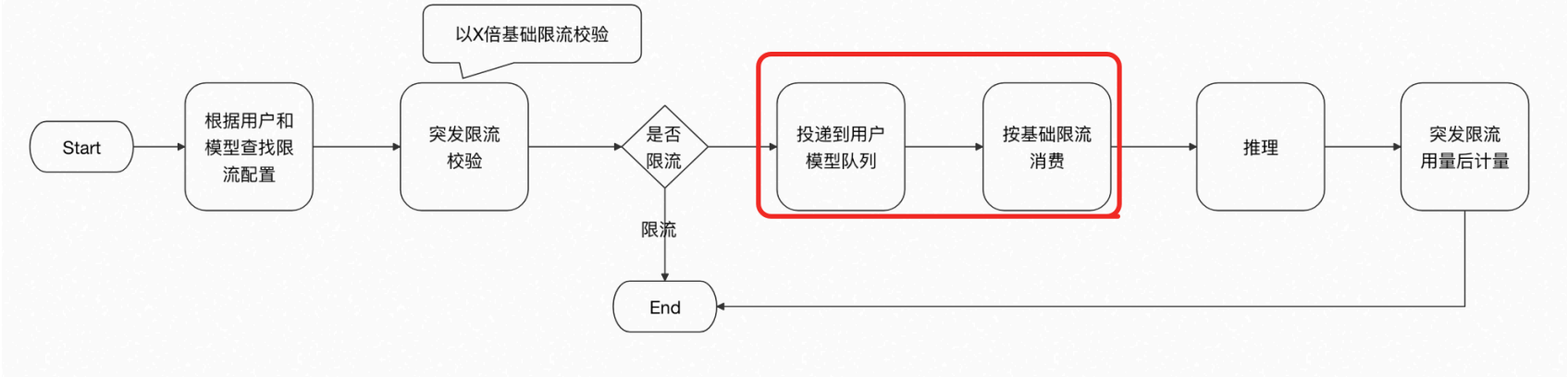
放行通知回到用户连接机器。



效果 10× 限流异常比例下降，用户可感知问题收敛，资源合池

PART 02 · 2.5 通用抽象

从百炼方案抽象出的四个关键点



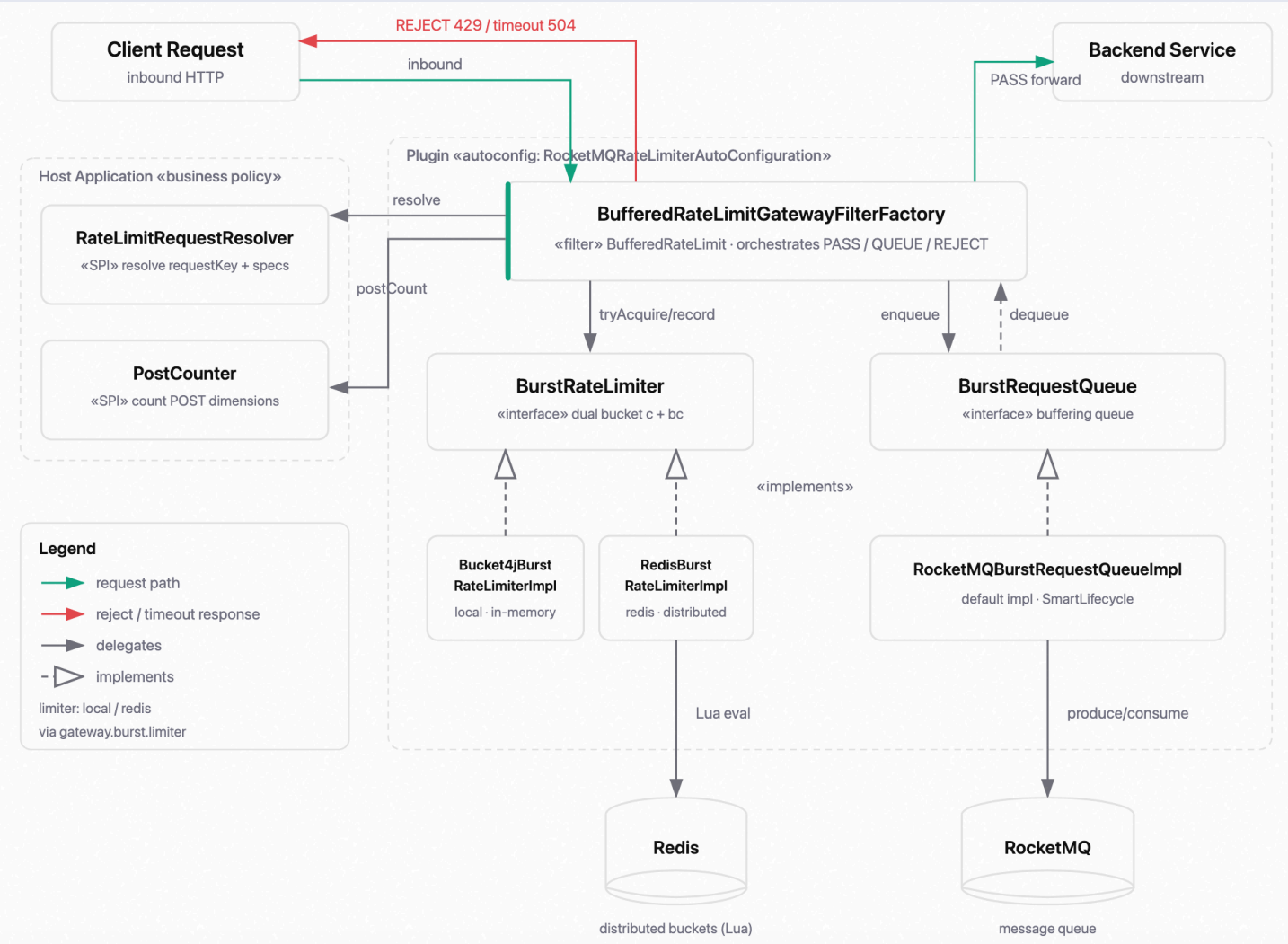
1 用户级限流配置
按用户 + 模型查找限流配置

2 突发限流识别
以 X 倍基础限流做突发校验

3 缓冲队列
超限请求投递到用户模型队列

4 放行及调用后处理
按基础限流消费，推理后计量

spring-cloud-gateway-rocketmq-ratelimiter



业务方只需实现 Resolver / PostCounter 两个 SPI，其余由插件自动装配

PART 02 · 2.7 关键能力

五项能力，开箱即用

1

削峰不阻塞

排队全异步，不拖垮网关事件循环

2

多维度同时限流

按用户、API、IP 等多维度原子管控

3

按结果计量

支持响应后按流量 / 行数计数

4

单机/分布式

内存版与 Redis 版可插拔切换

5

一键集成

Spring Boot Starter 交付

通用化的插件

THANK YOU

RocketMQ LiteTopic · 实践案例

LoongSuite Pilot

让每一个 AI Coding Agent
可观测、可评估、可审计

石木（陈扬）| 阿里云可观测团队

01

可观测性的盲区

AI Coding Agent 时代，我们答不出的问题

当 AI Coding Agent 成为日常工具

这些基本问题，我们无从回答：

? 团队每天消耗了多少 Token？

? 哪些任务适合 Agent，哪种任务适合哪个 Agent？

? Agent 输出不符预期时，中间经历了什么？

? Agent 修改了 30 个文件，完整链路是什么？

企业月投入数万到数十万，却无法量化“这笔钱值不值” → 经营问题

02

为什么这件事难做

困难来自三个层面

AI 编程 Agent 的可观测性：三层困难

Agent 行为天然难观测

一次任务 10+ 轮 ReAct 循环

每轮涉及模型调用、工具执行、

结果反思与决策

传统 MLT 无法还原决策流程

多 Agent 数据天然割裂

每个 Agent 的数据：

格式不同（JSON / SQLite / 文本）

存储位置、粒度、语义各异

无法规模化横向对比

端侧是可观测的盲区

主机 / 容器 → LoongCollector ✓

服务端框架 → 语言探针 ✓

开发者桌面 → 无覆盖 ✗

核心问题：如何在不改造 Agent 的前提下，从端侧获取完整、统一、可靠的行为数据？

03

1 分钟跑通全流程

从安装到效果展示

一行命令安装，数据自动采集

```
$ Terminal

curl -fsSL https://...loongsuite-pilot/installer.sh | bash -s -- install \
  --sls-endpoint "https://cn-xx.log.aliyuncs.com" \
  --sls-project "my-project" --sls-logstore "my-logstore" \
  --sls-ak-id "your-ak-id" --sls-ak-secret "your-ak-secret"
```

01

自动部署

下载 → 安装 Hook
→ 启动后台进程

02

静默采集

正常使用 Agent
Pilot 后台运行

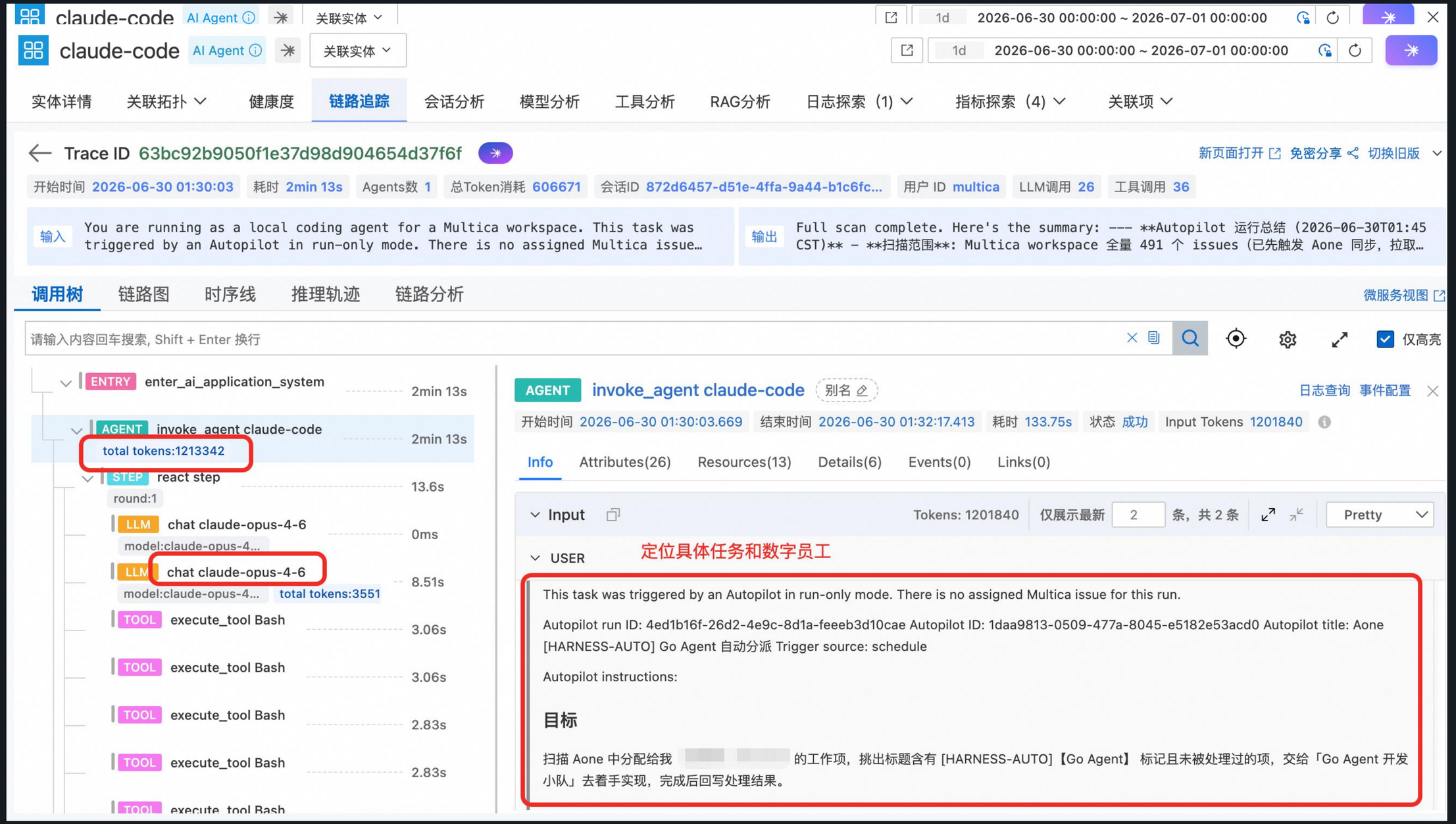
03

本地 Dashboard

loongsuite-pilot
monitor start

实际演示

Trace 数据使用案例 – Token 问题定位



04

LoongSuite Pilot 的解决思路

架构 + 五个关键设计选择

LoongSuite Pilot 架构总览

AI Coding Agents

IDE 历史文件 | SQLite 数据库 | Hook JSONL 日志 | Session 日志 | Patch 拦截输出 | 更多 ...

LoongSuite Pilot

Discovery + Deployment

- **AgentDiscoveryService** — fs.watch + 轮询发现
- **DeploymentManager** — 4 种策略自动注入
- **agents.d/*.json** — 声明式配置

Input Layer (InputManager)

- **HookInput** — JSONL 增量
- **IdeInput** — IDE 快照
- **SqliteInput** — rowid 游标
- **SessionInput** — 会话文件
- **CliForwarder** — CLI 转发

Processing

- **Normalization** — → AgentActivityEntry
- **Content Policy** — 按 Agent 控制粒度
- **Masking** — AK/Key/URI/私钥 脱敏
- **Git Enrichment** — 关联 repo / branch

MultiFlusher (并行扇出)

- **SLS Logstore** — 生产实时查询
- **本地 JSONL** — 零依赖调试
- **HTTP Endpoint** — 对接自建后端
- **OTLP Trace** — CMS 2.0 / Jaeger / Langfuse

Checkpoint

StateStore
读取偏移持久化
input-state.json

SnapshotStore
快照去重
snapshot-store.json

断点续采
不丢 · 不重

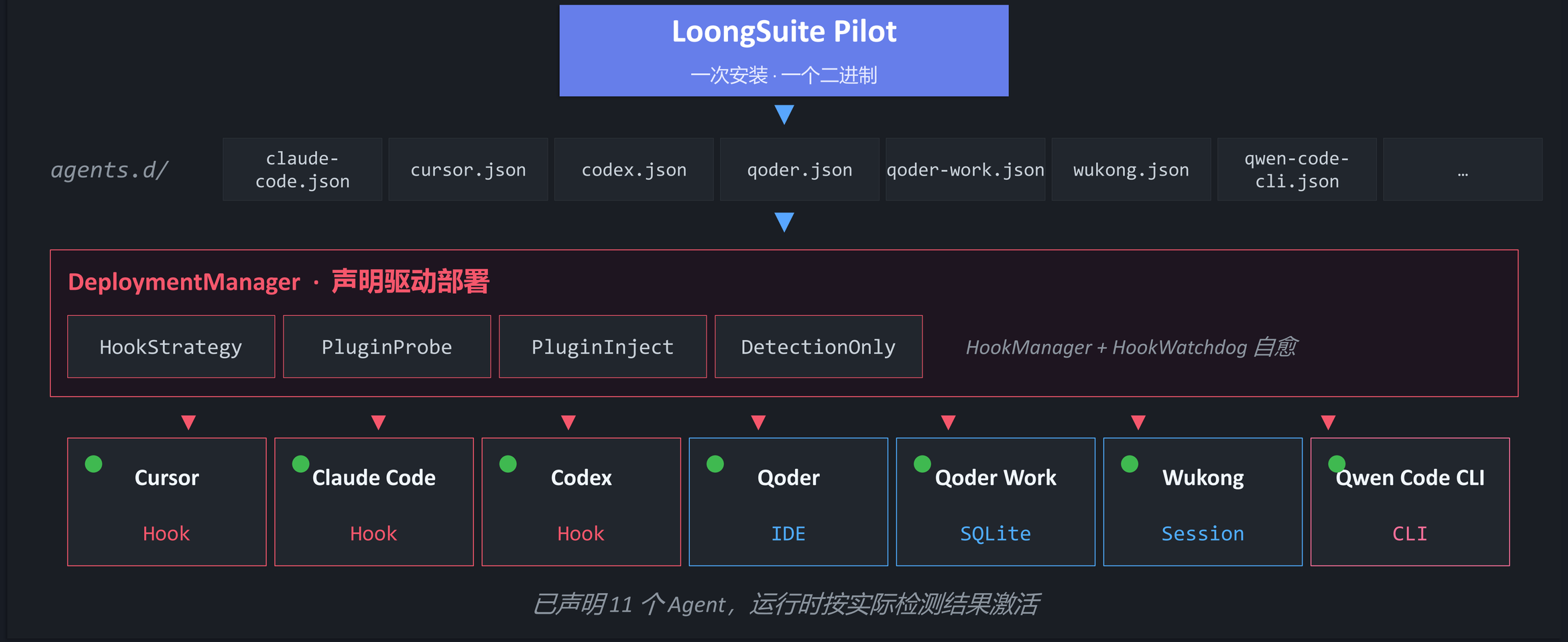
运维能力　HookWatchdog · AutoUpdater · Metrics · StatusBar · LogRetention

存储 & 分析后端

- **阿里云 SLS** — 实时查询分析
- **CMS 2.0 / Langfuse** — OTLP 链路追踪
- **本地 JSONL** — 零依赖落盘
- **自建后端** — HTTP 对接

选择一： ALL IN ONE 架构

一次部署，自动覆盖所有已安装的 AI Coding Agent



声明驱动 → 新增一个 Agent 适配 = 一个 JSON 声明 + 一个 Input 实现，无需改框架代码

选择二：多 Agent 采集矩阵

5 种采集基类 × 多种 Agent = 全覆盖

BaseHookInput

Hook JSONL 日志增量读取

BaseIdeInput

IDE 历史文件快照轮询

BaseSqliteInput

SQLite rowid 游标增量查询

BaseSessionInput

Session 文件轮询

BaseCliForwarder

CLI 遥测日志转发

Cursor

Hook 注入

Claude Code

Hook + Node.js 启动项注入

Codex

Hook + Node.js 启动项注入

Qoder

IDE 快照 + Hook + Session

Qoder Work

SQLite + Hook

OpenClaw / Gemini CLI / Cline ...

新增 Agent 只需：继承基类 + 编写 agents.d/.json 声明文件 + 实现数据转换方法*

选择三：统一语义规范

基于 OpenTelemetry GenAI Semantic / OTEL GenAI Util SDK



口径对齐

所有 Agent 使用相同字段名和语义定义

gen_ai.usage.input_tokens

gen_ai.session.id ...

层级化链路

还原 Agent 完整执行链路

每一轮 ReAct 循环的

思考→工具调用→推理

基础设施复用

接入新 Agent 时

看板、告警、分析查询

自动生效，无需重新开发

选择四 & 五：安全脱敏 + 多目标输出

灵活粒度，平衡观测与安全

采集粒度控制

按 Agent 类型配置是否采集消息内容、工具参数

敏感信息自动脱敏

云 AK → [ACCESSKEY_MASKED]

API Key → [APIKEY_MASKED]

数据库连接串 → [DATABASEURL_MASKED]

私钥 → [PRIVATEKEY_MASKED]

```
mask.mode: none | all | custom
```

多目标输出，不绑定后端

本地 JSONL	零依赖落盘，本地调试
SLS Logstore	生产级实时查询
HTTP Endpoint	对接自建后端
OTLP Trace	CMS 2.0 / Jaeger / Langfuse

并行扇出，单个目标失败不阻塞其他通道

05

数据能回答什么问题

四个以前回答不了的关键问题

有了数据之后：四个最重要的场景

AI 编程的 ROI 如何度量?

传统指标失效，需新维度：
任务完成率 / Token 效率比 / 人机协作比
→ 全链路 Trace 树 Top-down 排查

多 Agent 并存，选型靠什么?

不同 Agent 有不同的“甜蜜区”
统一 schema 横向对比：
Token 效率 / 成功率 / 时长

Token 花得多就是效果好吗?

Token 消耗与质量非线性
识别“Token 黑洞” = 发现能力瓶颈
→ 多维聚合 + 缓存命中率评估

AI Agent 的操作谁来审计?

Agent = 大规模拥有写入权限的非人类实体
行为分析 → 风险评分 → 会话追溯

每个场景背后，都需要完整、统一、可靠的 Agent 行为数据作为基础设施

Agent 选型实践 -- Claude Code vs QoderCLI

相同考题 · 相同准出条件 · Trace 数据后台自动采集，无人工干预

考生

Claude Code (opus-4-6) vs QoderCLI (Ultimate)

考题

8 道复杂编程任务 · 覆盖实现 / 修复 / 编译 / 并发 / 重构 / 迁移

准出

每题 22–32 个预写单元测试，必须全部 PASS

采集

LoongSuite-Pilot 后台自动采 Trace · 零埋点零改码

执行方式

claude --print / qodercli -p 非交互一次性执行

npx vitest run 自动验证

根据 session 提取数据分析生成报告

8 道考题矩阵

覆盖不同编程场景

每题预写 22–32 个测试作为准出线

#	类型	任务内容	单测数
T1	多模块实现	EventBus (核心+中间件+通配符+异步)	22
T2	跨文件 Bug 修复	4 文件中 12 个跨调用链 bug	32
T3	编译器实现	Markdown → HTML 转换器	32
T4	并发系统	DAG 调度 (拓扑排序+超时+失败级联)	26
T5	API 设计	链式 SQL 查询构建器 (不可变)	22
T6	重构+扩展	单体 Logger → 插件架构	28
T7	代码迁移	Callback → async/await	25
T8	代码生成	JSON Schema → TS 类型 + 校验器	26

评测结果：质量全部达标、效率各有所长

426 次单元测试 100% PASS · 效率差异全部由 Pilot 的 OTLP Trace 直接提取

质量维度

213 × 2 = 426

预写测试全部通过 · 双方 100% PASS · 质量打平

Token 消耗

2.03×

Claude Code 4.4M vs QoderCLI 2.1M · QoderCLI 省 51%

端到端速度

× 0.61

Claude Code 967s vs QoderCLI 1593s · Claude 快 39%

汇总对比（8 个任务合计）

数据来源：LoongSuite-Pilot OTLP Trace · 按 gen_ai.session.id 聚合

指标	Claude Code	QoderCLI	优势方
LLM 请求次数	78	66	QoderCLI
ReAct 轮次	78	66	QoderCLI
工具调用次数	70	75	Claude Code
Total Tokens	4.4M	2.1M	QoderCLI
LLM 总耗时	848.8s	1531.0s	Claude Code
端到端总耗时	966.9s	1593.0s	Claude Code

单任务效率均值

平均端到端	120.9s	199.1s	Claude 快 39%
平均推理	106.1s	191.4s	Claude 单次更快
平均 Token	545.3K	268.7K	QoderCLI 省 51%
平均 ReAct	9.8 轮	8.2 轮	QoderCLI 更精简

Trace 才能看到的两个洞察

T3 Markdown 编译器

QoderCLI 触发 max_tokens 截断续传耗时 812s，Claude 一气呵成 324s——finish_reasons=['max_tokens'] 只能在 Trace 里看到

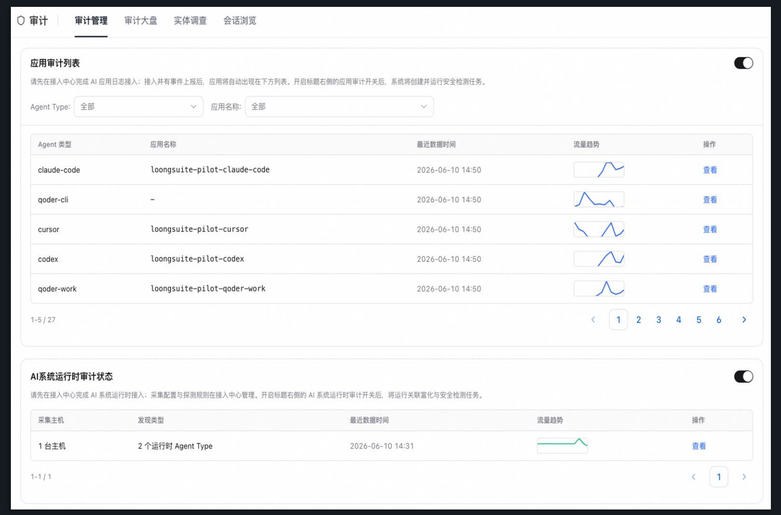
T6 插件架构重构

QoderCLI 24 轮小步迭代 389s，Claude 16 轮 165s——Trace 的 STEP 序列揭示"先理解后动手" vs "写-测-改"策略差异

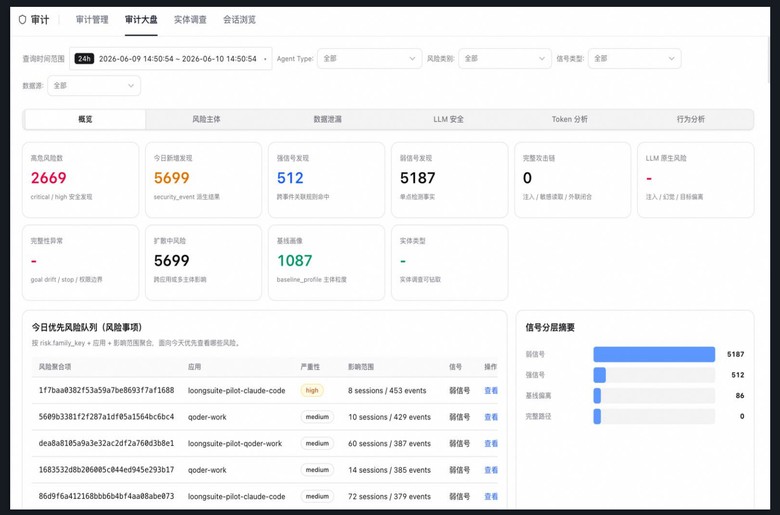
结论

Agent 选型不再靠感觉——每一次 LLM 调用、每一次工具使用、每一轮 ReAct 都成为可量化、可对比、可追溯的数据

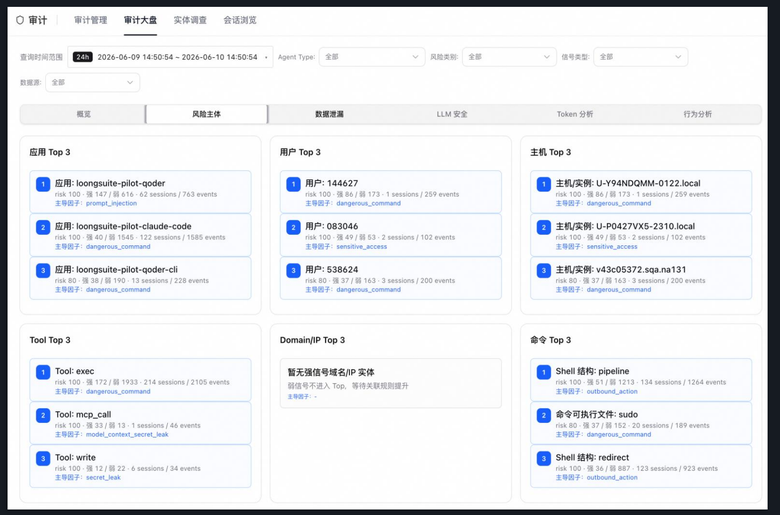
安全审计：六层体系从管理到追溯



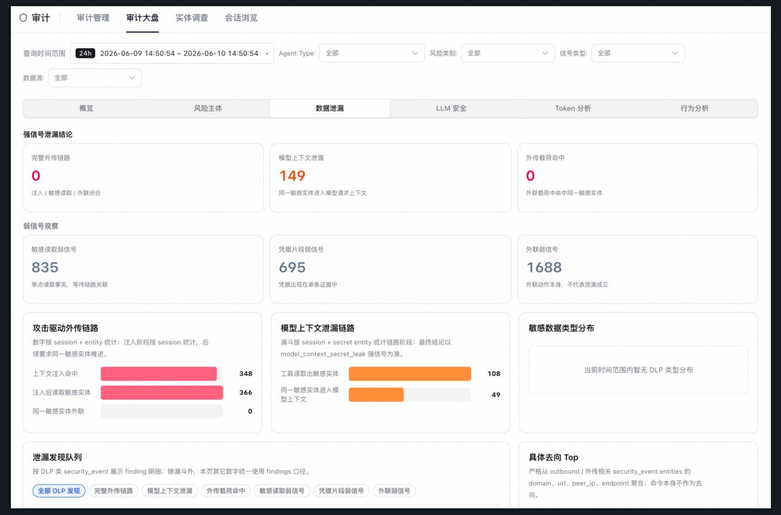
① 审计管理总览



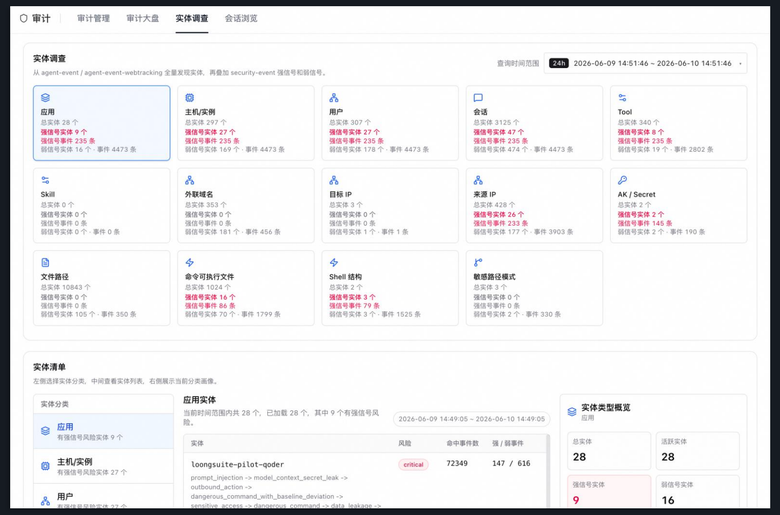
② 风险态势大盘



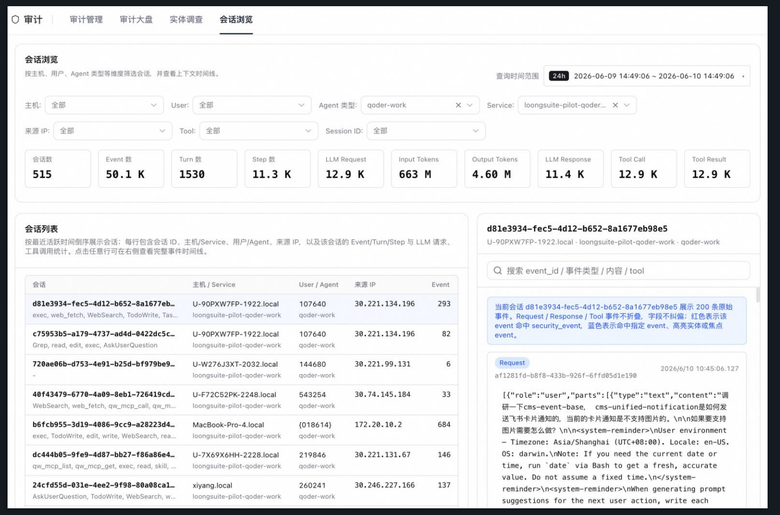
③ 风险主体定位



④ 数据泄露分析



⑤ 实体关联调查



⑥ 会话级风险追溯

06

开源与社区

LoongSuite Pilot 正式开源

LoongSuite 可观测生态定位

端侧 AI 编程 Agent

Cursor | Claude Code | Codex

服务端 AI Agent

Dify | AgentScope | LangChain

基础设施

主机 | K8s

LoongSuite Pilot

端侧 AI 编程 Agent 数据采集

Python / Go / Java Agent

服务端框架无侵入采集

LoongCollector

主机 / 容器 / eBPF

多协议输出：OTLP · SLS Protocol · HTTP · 本地文件

阿里云 SLS

实时查询

CMS 2.0

链路追踪

Grafana

可视化

ES

全文检索

LoongSuite Pilot 补齐了 AI Agent 可观测在端侧的一块拼图

ROADMAP & CONTRIBUTING

下一步计划

更多 Agent

新agent， 露头即秒

开箱即用分析看板

效率度量 + 成本分析 Dashboard

社区 Agent 贡献模板

标准化的第三方 Agent 适配流程

三种参与方式

提 Issue

报 Bug、提新 Agent 适配需求

贡献 Agent 适配

参考 agents.d/ 声明文件和已有实现

分享实践

把使用经验和分析洞察分享到社区

钉钉群

微信群

端侧可观测，需要全社区一起建。

THANKS

可观测性是让 AI Agent 从“能用”走向“好用”的
关键基础设施

github.com/alibaba/loongsuite-pilot

石木（陈扬）| 阿里云可观测团队

AGENTTEAMS · 数字员工实践

基于 AgentTeams 的数字员工落地实践

一项职能=一支 Agent 团队：可复制的数字员工工厂

- 批量构建
- 标准交付
- 持续运营

不是单点机器人，而是方法论与工程化实践

LEADER — WORKER 协同

背景 · 为什么需要数字员工

研发组织的“职能类工作”之痛

01 · 重复消耗

大量工时被重复劳动吞噬

值班答疑、工单处理、社区 Issue、取数提单，**日复一日消耗研发工时**。

02 · 频繁打断

碎片任务打断主线工作

碎片化请求不断插入，工程师**难以持续专注**于核心研发主线。

03 · 专家单点

关键知识集中在少数人

专家成为单点瓶颈，权限只能**“全有或全无”**，无法安全分级下放。

04 · 7×24 压力

响应压力不分昼夜

社区与告警**不分时区、不分工作日**，团队被迫 7×24 待命响应。

核心破题思路

一项职能 = 一支 Agent 团队

> 我们不是在做一个**孤立的机器人**，而是把一项职能完整交给一支**可协同、可治理、可进化**的 Agent 团队。

可协同

Leader × Worker 分工，规划与执行各司其职

可治理

权限矩阵、审计追踪与安全护栏兜底

可进化

经验沉淀持续回流，越用越懂业务

从机器人到数字员工团队：范式升级

传统脚本 / 单点机器人

按指令执行的“工具”

- ✕ **固定流程：**只能按预设脚本执行，无法应变
- ✕ **单一技能：**一个机器人只会做一件事
- ✕ **无法拆解长任务：**复杂链路一断即停
- ✕ **出错难追溯：**黑盒执行，定位问题靠人工
- ✕ **经验不沉淀：**每次执行都从零开始

VS

AgentTeams 数字员工团队

推荐

主动交付结果的“团队”

- ✓ **多角色协作：**Leader 规划 + Worker 专业执行
- ✓ **任务拆解与并行：**长任务自动分解、同步推进
- ✓ **按需调用工具与技能：**动态编排，灵活应变
- ✓ **全程可追溯：**计划、执行、报告透明可审计
- ✓ **经验资产化：**成果沉淀复用，持续进化

→ **范式升级：**从「执行一个动作」的自动化，走向「交付一件完整事情」的团队式智能

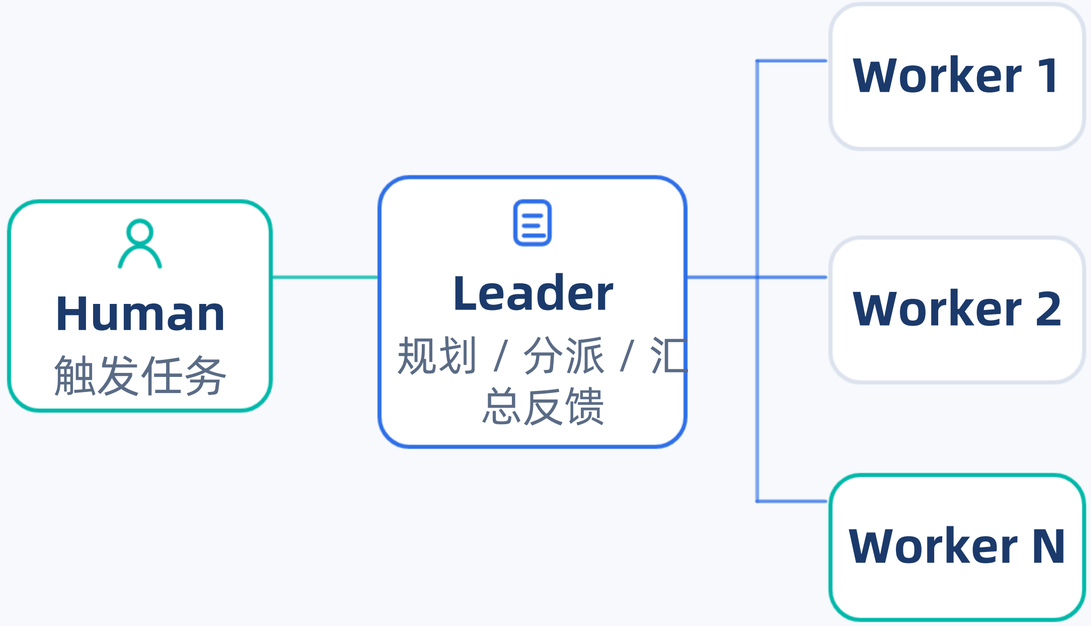
AgentTeams 协同模型：Leader × Worker

Team = Human + Leader + N Workers

Human 触发，Leader 规划、分派并汇总反馈，Worker 专业执行

Team 协同结构

Leader 和 Worker 都是 Agent



Leader × Worker 分工

Leader 负责中枢决策

理解意图、制定 Plan、选择工具与 Worker，并统一管理上下文。

Workers 专业化执行

按领域拆分子任务，分别调用知识、系统、工具完成可验证动作。

反馈归 Leader 汇总

Worker 只回传执行结果，由 Leader 汇总为报告或操作结论，再反馈给 Human。

规模化复制全景

数字员工团队矩阵全景

从样板间到 Team 与 Agent，把客户场景沉淀为可配置、可治理、可复用的数字员工工厂。

7 / 25

样板间矩阵
7 个样板间
25 个子样板间

49 / 75

Team 与 Agent
49 个 Team，承载 75 个 Agent

300+

覆盖用户
面向业务用户形成使用闭环

18+1

管理员治理
分层运营，保障复制质量



样板间是复制单元
把流程、角色与知识固化为可复用配置

Team 承载职能
多 Agent 协同，支撑端到端任务交付

治理保障复制
统一管理员体系，沉淀运营与质量标准

工程视角：样板间代码组织形式

AI Agents 团队 代码仓库 实际仓库

```
ai-agent-teams/
├── skills/ ← 仓库级公共能力池
└── 样板间2：值班答疑/
    └── observe-oncall/
        ├── skills/ ← 样板间级 Skills
        ├── observe-oncall-leader/
        │   ├── config/
        │   │   ├── SOUL.md 身份与边界
        │   │   ├── AGENTS.md 协作契约
        │   │   ├── MEMORY.md 长期记忆
        │   │   ├── BOOTSTRAP.md 启动自检
        │   │   └── HEARTBEAT.md 运行心跳
        │   ├── skills/ ← Agent 本地 Skills
        │   └── ref_skill.json ← 声明式引用清单
        ├── observe-oncall-cmsv1-agent/
        ├── observe-oncall-cmsv2-agent/
        └── observe-oncall-cloudlens-agent/
```

五件套配置规范

SOUL 身份 | AGENTS 协作 | MEMORY 记忆 | BOOTSTRAP 启动 | HEARTBEAT 心跳

配置即契约：我是谁、和谁协作、记住什么、如何稳定运行

三层 Skill 架构

仓库级 skills — 跨项目通用 | 样板间级 skills — 场景模板 | Agent 本地 skills — 角色专属

ref_skill.json 声明引用，打包时就近向上注入

先本地验证 → 沉淀样板间 → 进入仓库级共用

配置即代码 · Agent 可复制 · Skill 可复用

落地流程：从定义到进化的闭环

数字员工全生命周期六步走：交付不是终点，运行经验持续回流、持续进化

01

定义

五件套配置规范 + 技能编写，定义身份、契约与能力边界

02

规范检查

路径 / 映射 / 凭证 / Git 四项审计，发布前逐项把关

03

打包发布

统一打包 + 模版发布，两步走灰度防止打穿全量

04

批量建站

五阶段自动化 + AI 语义审核 + 断点续连，规模化交付

05

运行监控

拨测探活 + 响应时延跟踪，持续掌握运行健康度

06

回流进化

运行时变更回流 Git，经验沉淀反哺定义与技能

场景一痛点：开源社区维护

案例场景描述

Apache RocketMQ 是阿里开源的分布式消息中间件，GitHub 上持续产生大量 Issue 和 PR。核心维护团队有限，需要自动化方案保障**响应速度**与**代码质量**。

关键挑战：用自动化放大有限维护人力。

传统方式的痛点

01	**Issue 响应周期长**	人工确认、定位、修复方案平均 3-5 个工作日
02	**Code Review 瓶颈**	单个 PR 需阅读 diff、上下文和安全/兼容性，耗时 30-60 分钟
03	**跨仓库一致性难保证**	rocketmq 与 rocketmq-clients 协议同步易遗漏
04	**新贡献者 CI 审批延迟**	首次贡献需 maintainer 手动 approve，受时区影响
05	**社区活跃度受影响**	响应慢导致贡献热情下降，Issue 堆积



开源社区维护数字员工团队

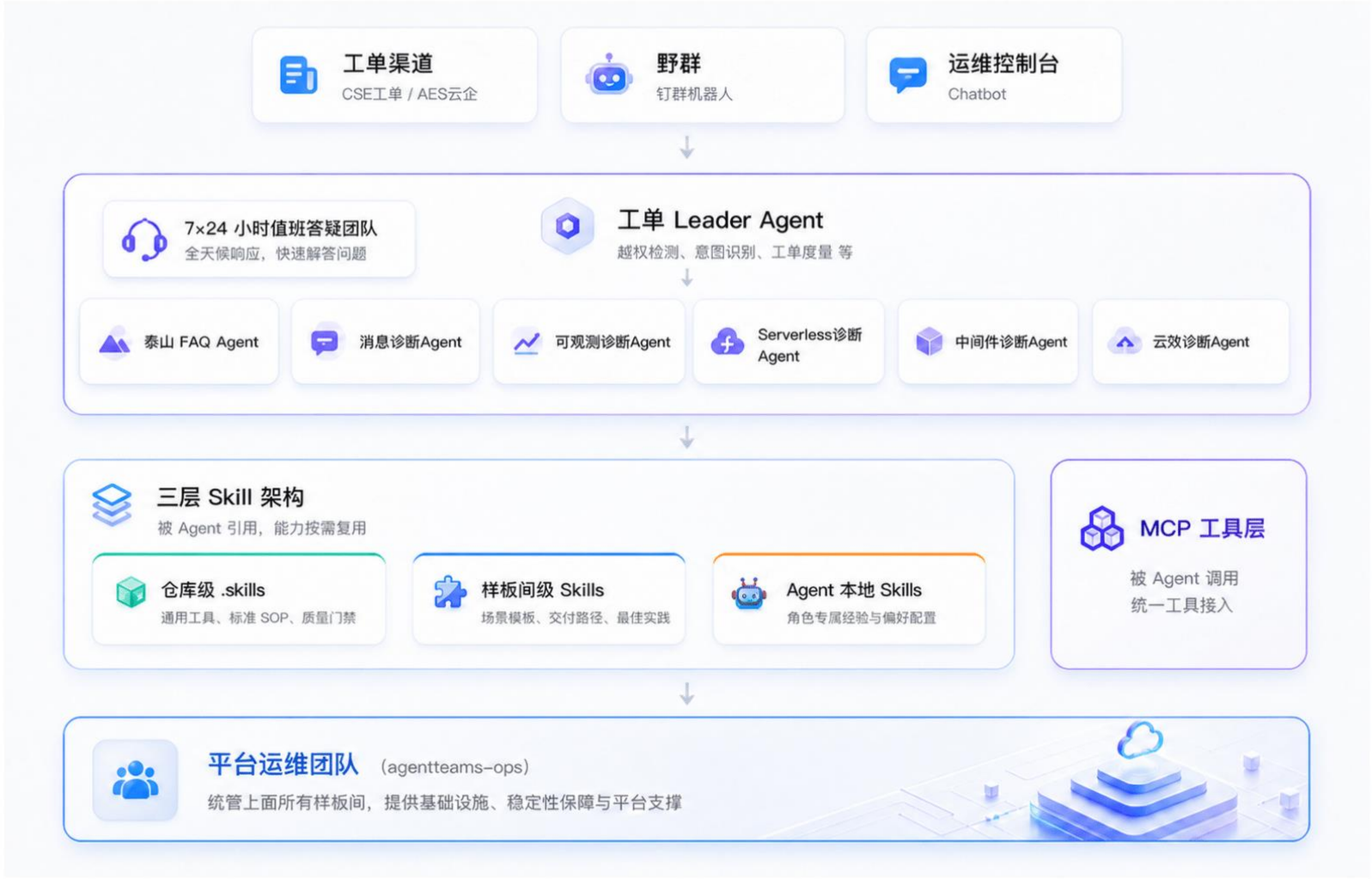
全链路闭环：
- issue → Spec
- 人工：{/approve}
- PR Review→Merge

效率提升
首响 3-5天→1小时内。

质量改善
Review 自动且覆盖率 100%。

成本优化
节省约 60% 维护时间。

创新价值
覆盖 10+ 个社区 / 25+ 个仓库。



覆盖 9 个产品、22 个诊断 Agent、86 个诊断业务技能

场景二：Oncall 值班与专业分诊

01 多入口接入
工单、群聊、控制台统一进线。

02 Leader 分诊
识别意图并派发专业 Agent。

03 专业诊断
FAQ、可观测、云产品协同诊断。

04 能力复用
Skill 与 MCP 工具层稳定答疑。

协同结果
专家能力编排可复用，可度量

工单度量：工单 AI 覆盖金字塔

闭环率和工单逃逸率看 Agent 诊断能力；**场景逃逸率**看能力覆盖缺口。

AI 闭环率

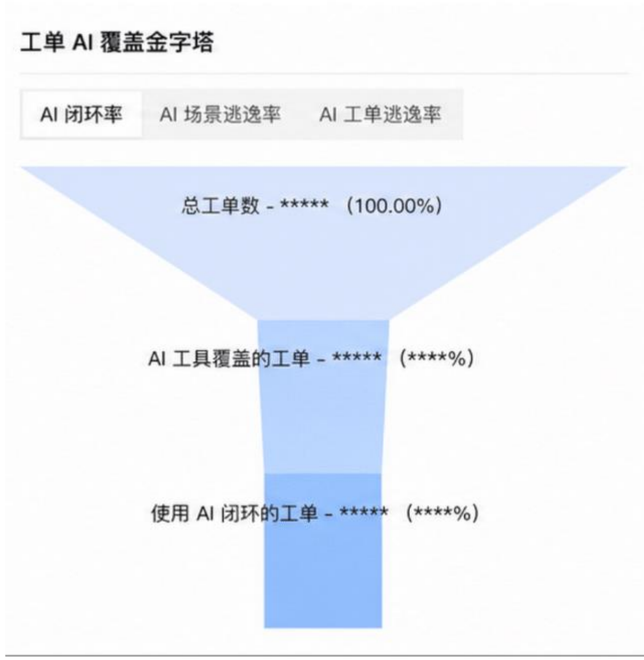
场景已覆盖时，诊断 Agent 无需人工升级即可完成闭环的能力。

AI 工单逃逸率

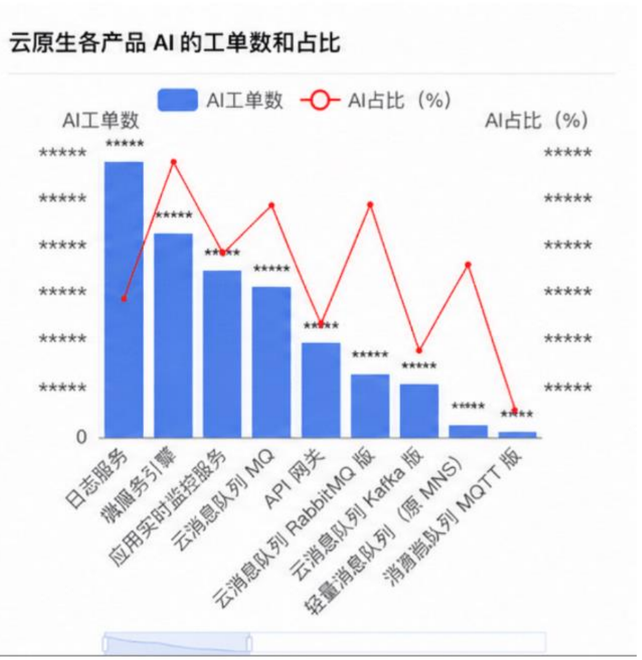
场景已覆盖且已调用 AI，但仍需升级人工，反映诊断 Agent 能力短板。

AI 场景逃逸率

诊断场景尚未覆盖，需补充知识库、诊断工具或专项能力。







场景三：应急数字人协同处置

一切围绕缩短 **MTTR**：应急指挥 Leader 统筹调度，四个 Worker 并行协作、闭环处置

衡量指标：MTTA/MTTR 改善 / 根因定位命中率 / 恢复验证覆盖率

应急指挥 Leader

接警分类 · 任务分派 · 全程跟踪闭环

响应红线

- 5分钟内分类分派
- P0/P1 即时通知
- 超时自动升级

01

信息同步

统一事件口径，实时播报进展，拉通全部干系人

02

根因分析

聚合日志与指标，快速收敛疑点，定位故障根因

03

恢复处置

执行预案与回滚，优先恢复业务，动作全程留痕

04

恢复验证

核验指标回归基线，确认业务恢复，闭环关闭事件

MTTR 链路 故障发现 → 分类分派 → 并行协作 → 恢复验证 → 事件关闭

场景四：经营分析团队与自然语言问数

自然语言问数 01

业务同学直接提问收入、客户、目标达成类问题，自动生成查询语句取数，无需等排期。

示例：“本季度华东区收入目标达成情况如何？”

口径铁律 02

指标口径版本化沉淀、持续更新，全团队统一口径，避免“同名指标、各说各数”。

一个指标，只有一个权威口径

准确性和效率的平衡 03

PTC（Programmatic Tool Calling，编程式工具调用）：查询一次沉淀为脚本。产品经营日报、周报查数等。

数据源未接入就如实说明查不到，绝不编造数字；每个结论都可回溯到取数来源。

分析协作 04

问数 Leader 统一调度多个分析 Worker 并行取数与分析，共同支撑业务工作台。

1 个 Leader + N 个 Worker 协同作战 | 衡量指标：问数命中率 / 口径匹配准确率 / 取数耗时下降

你敢让 AI 做生产变更吗？

AgentTeams OPS：“使用 AI 团队 运维 AI 团队”

误操作　越权　不可追溯

LLM 是“概率性执行者”

输出天然带随机性。真实教训：一次调用**全量覆盖接口**，导致成员被**误删**。

Agent 是自我进化的“活体”

运行时自改配置，会与版本库**脱钩**，持续产生**配置漂移**。

答案，是把安全判断搬出模型——不由 AI 自己说了算。

见下一页 →

可信护栏：可解释、可追溯、可拒绝

衡量指标：权限覆盖率 / 拦截次数 / 审计完整率

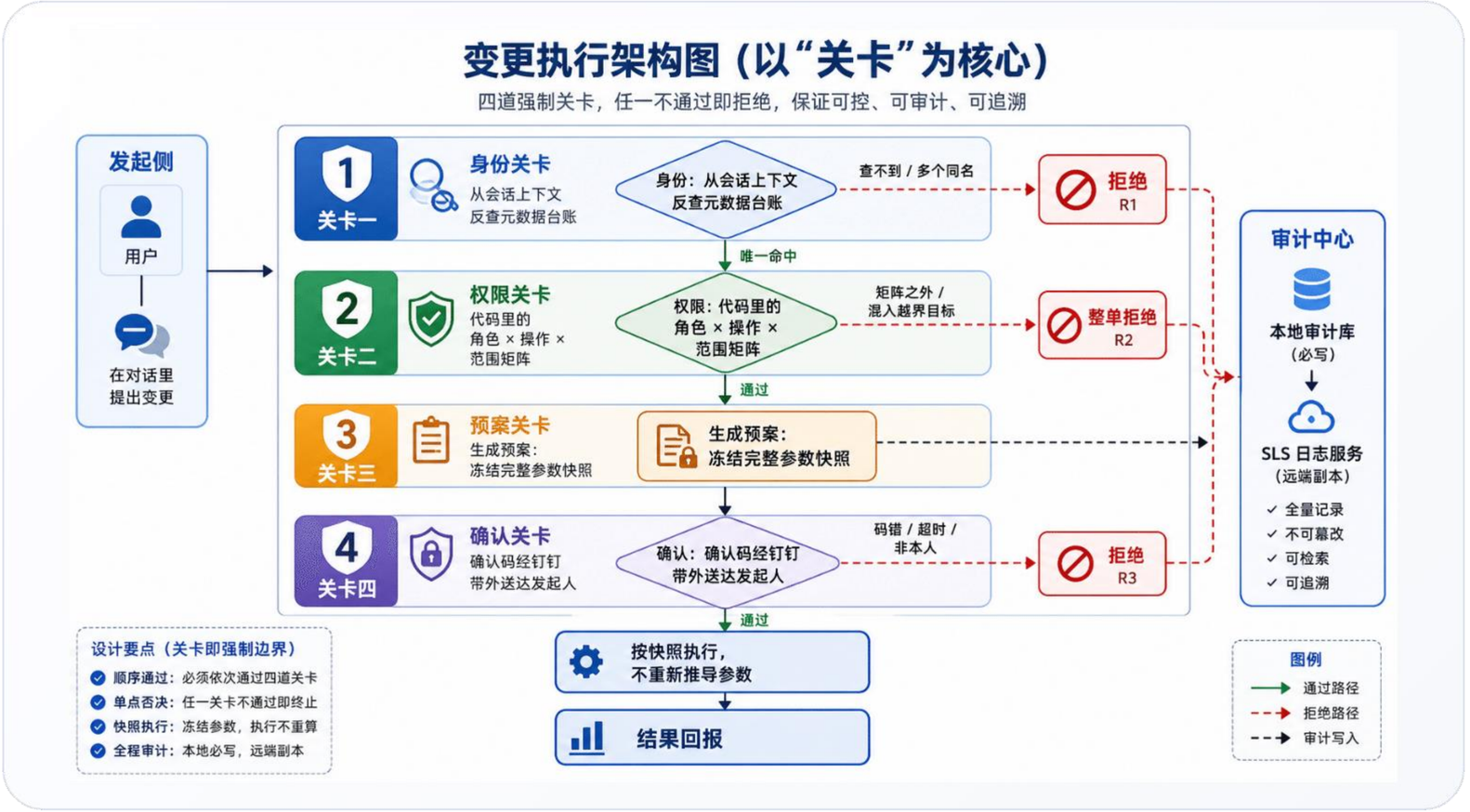
执行前门禁

把四道关卡压进外部治理链路，模型只给建议，代码决定能否执行。

- **身份**
  调用者与目标资源一致
- **权限**
  按矩阵裁决可做什么
- **预案**
  影响面与回滚先写清
- **确认**
  确认内容绑定执行内容

AgentTeams Ops 团队 四道关卡治理链路

四道关卡把高危操作纳入可裁决、可审计、可拒绝的外部治理链路。



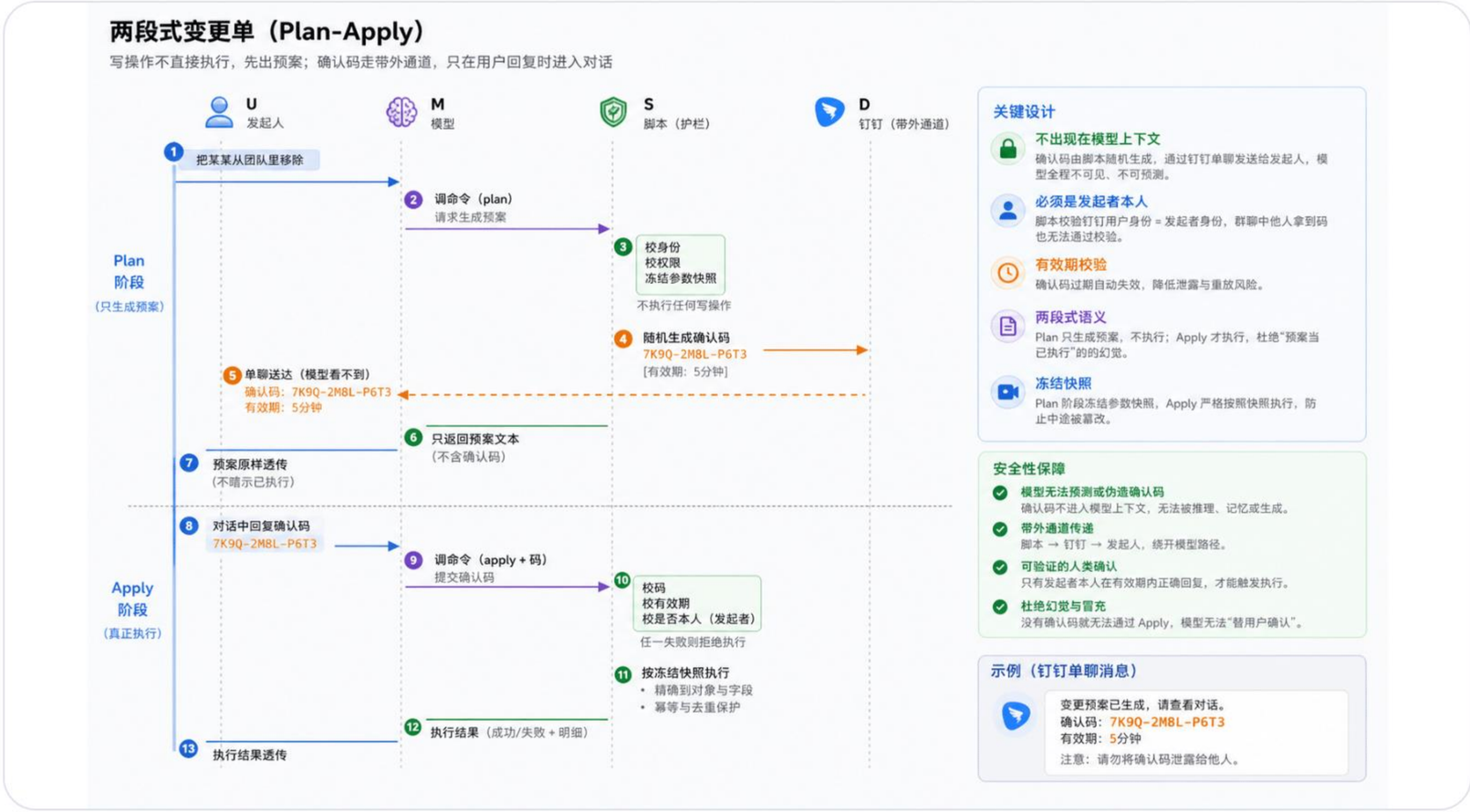
- **可解释**
  变更范围、理由可见
- **可追溯**
  全程留痕，记录SLS
- **可拒绝**
  风险拦截，快速失败

高危操作治理：两段式确认机制

协议只留三件事

1. 先冻结 **payloadhash**，锁定变更意图。
2. **带外随机码**经钉钉发送，原通道回填。
3. 通过 **本人 + TTL + hash** 校验后执行。

确认链路绑定意图、确认人、时效与 payload。

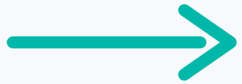


随机码，不会出现在LLM上下文中，防止模型绕过审计执行任何高危操作。

用 AI 运维 AI：从管理员瓶颈到自助运营

1 名系统管理员

人工执行·单点瓶颈



18 人安全自助运营

授权自助·审计闭环

统一管理

运维数字员工统一承接生产资源入口

分级授权

权限按角色下放，让业务团队自助完成

审计闭环

每次操作留痕，可解释、可追溯、可复核

角色升级

管理员从人工执行者升级为规则制定者

这本身就是数字员工的最佳实践 ——Dogfooding：用 AI 运维 AI

可观测与自进化：数字员工也要被运营

上线只是起点：数字员工需要持续拨测、审计回流与版本化发布，才能长期可信。



可观测

除了 Loongsuite 指标采集 + ProbeAgent 探针，保障Agent可用性。



进化回流

Badcase 与经验进入 MR、Review、Release，形成可回滚的工程闭环。

落地价值：从工具提效 到 AI Native 组织能力升级

人力释放

职能类打断性工作移交数字员工，真人回归主线

7×24 在线

黑灯工厂、值班答疑、github社区、应急全天候第一时间响应

规模化复制

样板间模式，新团队接入边际成本递减

中心主张



数字员工融入日常协作，能力沉淀为组织资产

可信可控

权限矩阵、两段式确认、全量审计，零越权零误删

知识资产化

口径、SOP、经验版本化沉淀，人员流动不断层

规模化验证

7 样板间　　**49** 接入团队

75 数字员工　　**1→18** 问题自助

THANKS

欢迎交流 / Q&A